ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



BÁO CÁO BÀI TẬP LỚN
KHO DỮ LIỆU VÀ HỆ HỖ TRỢ QUYẾT ĐỊNH

KHO DỮ LIỆU GIAO THÔNG VÀ DỰ BÁO TẮC NGHẼN

Giảng viên hướng dẫn: Bùi Tiến Đức

Nhóm - Lớp L01 & L02 - HK251

Báo cáo cuối kỳ - Ngày nộp: 12/12/2025

STT	Họ và tên	MSSV	Ghi chú
1	Nguyễn Anh Khoa	2211612	L01
2	Nguyễn Xuân Hy	2211409	L02
3	Võ Ngọc Tú	2213857	L02
4	Trần Tuấn Kha	2211418	L01

Thành phố Hồ Chí Minh, tháng 12 năm 2025

Mục lục

1 Giới thiệu về Kho dữ liệu và Hệ hỗ trợ quyết định **3**
1.1 Kho dữ liệu (Data Warehouse) . . . 3
1.1.1 Định nghĩa và đặc trưng chính . . . 3
1.1.2 Kiến trúc và mô hình dữ liệu . . . 3
1.1.3 Công nghệ và công cụ . . . 4
1.1.4 Lợi ích và thách thức . . . 5
1.2 Hệ hỗ trợ quyết định (Decision Support System - DSS) . . . 5
1.2.1 Định nghĩa và thành phần . . . 5
1.2.2 Phân loại DSS . . . 6
1.2.3 Quy trình hoạt động . . . 6
1.2.4 Công nghệ hỗ trợ . . . 6
1.2.5 Ứng dụng và lợi ích . . . 7

2 Tổng quan vấn đề và mô tả bài toán **8**
2.1 Tổng quan về vấn đề tắc nghẽn giao thông . . . 8
2.2 Mục tiêu của dự án . . . 8
2.3 Vai trò của Kho dữ liệu (DW) và hệ hỗ trợ ra quyết định (DSS) trong bài toán giao thông . . . 9
2.4 Nguồn dữ liệu . . . 9
2.4.1 Tổng quan nguồn dữ liệu . . . 9
2.4.2 Cấu trúc dữ liệu . . . 10
2.4.3 Đặc điểm dữ liệu . . . 11
2.4.4 Phân tích tổng quan nguồn dữ liệu . . . 12

3 Thu thập và Tiền xử lý dữ liệu **13**
3.1 Khái niệm tiền xử lí dữ liệu . . . 13
3.2 Phân tích cột . . . 13
3.3 Xử lý giá trị thiếu (N/A) . . . 15
3.4 Chuẩn hóa dữ liệu . . . 17

3.5 Thu thập dữ liệu vào Staging 19
3.6 Xây dựng kho dữ liệu 21

4 Phương pháp nghiên cứu và mô hình hoá 24
4.1 Phương pháp 1: Tiếp cận khám phá và phân tích trực quan 24
4.1.1 Ma trận tương quan (Correlation Analysis) 26
4.1.2 Phân tích phân phối (Distribution Analysis) 28
4.2 Phương pháp 2: Tiếp cận khai phá dữ liệu và dự báo 30
4.2.1 Cơ sở lý thuyết 30

5 Kết quả 38
5.1 Đánh giá kết quả mô hình 38
5.1.1 XGBoost 38
5.1.2 MLP 39
5.1.3 Giao diện Streamlit 45

6 Kết luận và hướng phát triển 48
6.1 Kết luận 48
6.2 Hạn chế 48
6.3 Hướng phát triển 49

7 Ràng buộc bổ sung và hạn chế thực tế 50
7.1 Ràng buộc về độ trễ thời gian thực (Real-time Latency Constraint) . . 50
7.2 Tính đầy đủ và Tin cậy của hạ tầng cảm biến (Sensor Reliability & Availability) 50
7.3 Rào cản về Bảo mật và Quyền riêng tư (Privacy & Regulatory Compliance) 51

Tài liệu tham khảo 52

1 Giới thiệu về Kho dữ liệu và Hệ hỗ trợ quyết định

1.1 Kho dữ liệu (Data Warehouse)

Kho dữ liệu là hệ thống cơ sở dữ liệu tích hợp quy mô lớn, được thiết kế đặc biệt để hỗ trợ các hoạt động phân tích dữ liệu và ra quyết định quản lý cấp cao, khác biệt rõ rệt với các hệ thống giao dịch (OLTP) thông thường.

1.1.1 Định nghĩa và đặc trưng chính

Theo Bill Inmon - "cha đẻ"của Data Warehouse, đây là một tập hợp dữ liệu chủ thể định hướng (subject-oriented), tích hợp (integrated), không thay đổi theo thời gian (non-volatile), và hỗ trợ quyết định (decision support). Các đặc trưng nổi bật bao gồm:

- **Tích hợp (Integrated):** Kết hợp dữ liệu từ nhiều nguồn dị thể như hệ thống OLTP, file log, API bên thứ ba, dữ liệu streaming, đảm bảo tính nhất quán về định dạng, đơn vị đo lường và quy ước đặt tên.
- **Chủ thể định hướng (Subject-oriented):** Tập trung vào các chủ đề kinh doanh cụ thể (ví dụ: giao thông, bán lẻ, tài chính) thay vì các quy trình hoạt động, phục vụ phân tích sâu theo từng lĩnh vực.
- **Không thay đổi theo thời gian (Non-volatile):** Dữ liệu một khi được tải vào warehouse sẽ không bị xóa hoặc sửa đổi, tạo ra lịch sử đầy đủ để phân tích xu hướng dài hạn.
- **Hỗ trợ quyết định (Decision Support):** Tối ưu cho truy vấn phức tạp, phân tích đa chiều (OLAP), và báo cáo tổng hợp với khối lượng dữ liệu lớn (terabytes đến petabytes).

1.1.2 Kiến trúc và mô hình dữ liệu

Để tối ưu hóa hiệu suất phân tích, Data Warehouse thường áp dụng các mô hình schema sau:

- **Star Schema:** Bảng Fact (số liệu định lượng) ở trung tâm, bao quanh bởi các bảng Dimension (thông tin mô tả). Ưu điểm: truy vấn nhanh, dễ hiểu.
- **Snowflake Schema:** Dimension được chuẩn hóa thành nhiều bảng con, tiết kiệm không gian lưu trữ nhưng phức tạp hơn về truy vấn.
- **Galaxy/Constellation Schema:** Nhiều Fact tables chia sẻ Dimension chung.

Quy trình xây dựng theo mô hình ETL/ELT:

- **Extract:** Trích xuất dữ liệu từ nguồn (databases, files, APIs).
- **Transform:** Làm sạch, chuyển đổi, tổng hợp dữ liệu (data cleansing, integration, aggregation).
- **Load:** Tải vào Staging Area, sau đó vào Data Warehouse chính và Data Marts chuyên biệt.
- **Metadata Management:** Quản lý siêu dữ liệu để hỗ trợ truy vấn và bảo trì.

1.1.3 Công nghệ và công cụ

Các công nghệ cốt lõi hỗ trợ triển khai Data Warehouse bao gồm:

- **Storage:** RDBMS (Oracle, SQL Server), columnar databases (Amazon Redshift, Snowflake), NoSQL cho dữ liệu không cấu trúc.
- **ETL Tools:** Informatica, Talend, Apache NiFi, Microsoft SSIS.
- **OLAP:** Cube structures cho phân tích đa chiều.
- **Big Data Integration:** Hadoop, Spark cho xử lý dữ liệu lớn.

1.1.4 Lợi ích và thách thức

Việc triển khai Data Warehouse mang lại nhiều lợi ích quan trọng nhưng cũng đối mặt với một số thách thức:

- **Lợi ích:** Phân tích lịch sử toàn diện, hiệu suất truy vấn cao, hỗ trợ BI tools, tính nhất quán dữ liệu.
- **Thách thức:** Chi phí cao, thời gian triển khai dài, cần kỹ năng ETL phức tạp, bảo trì metadata.

1.2 Hệ hỗ trợ quyết định (Decision Support System - DSS)

Hệ hỗ trợ quyết định là hệ thống thông tin tương tác, máy tính hóa, sử dụng dữ liệu, mô hình, và kiến thức chuyên gia để hỗ trợ quá trình ra quyết định bán cấu trúc và không cấu trúc trong tổ chức.

1.2.1 Định nghĩa và thành phần

DSS được thiết kế để hỗ trợ các quyết định phức tạp mà không có quy trình cố định, kết hợp giữa dữ liệu lịch sử, mô hình dự báo, và kinh nghiệm con người. Các thành phần cốt lõi:

- **Cơ sở dữ liệu (Database):** Thường là Data Warehouse hoặc Data Mart chứa dữ liệu đã được tích hợp và làm sạch.
- **Mô hình phân tích (Model Base):** Bao gồm OLAP (Online Analytical Processing), thống kê, mô phỏng, machine learning, optimization algorithms.
- **Giao diện người dùng (User Interface):** Dashboard tương tác, báo cáo động, visualization tools (Tableau, Power BI).
- **Quản lý kiến thức (Knowledge Base):** Quy tắc chuyên gia, AI inference engines.
- **Công cụ giao tiếp:** Hỗ trợ làm việc nhóm, collaborative decision making.

1.2.2 Phân loại DSS

DSS có thể được phân loại theo nhiều cách khác nhau, với các loại chính sau:

- **Data-driven DSS:** Tập trung vào kho dữ liệu lớn, hỗ trợ truy vấn ad-hoc và phân tích xu hướng.
- **Model-driven DSS:** Sử dụng mô hình toán học, simulation, what-if analysis cho lập kế hoạch.
- **Knowledge-driven DSS:** Áp dụng AI, expert systems, rule-based reasoning.
- **Document-driven DSS:** Tìm kiếm, phân tích văn bản và tài liệu không cấu trúc.
- **Communication-driven DSS:** Hỗ trợ nhóm qua web conferencing, groupware.

1.2.3 Quy trình hoạt động

Quy trình hoạt động của DSS thường bao gồm các bước sau:

- **Thu thập thông tin:** Từ Data Warehouse và nguồn real-time.
- **Phân tích và mô phỏng:** Áp dụng models để dự báo scenarios.
- **Trình bày kết quả:** Visualization, recommendations.
- **Hỗ trợ tương tác:** Người dùng điều chỉnh parameters, test hypotheses.
- **Ra quyết định:** Kết hợp insight từ DSS với kinh nghiệm con người.

1.2.4 Công nghệ hỗ trợ

Các công nghệ hiện đại đang được sử dụng rộng rãi trong DSS bao gồm:

- **Analytics:** R, Python, SAS cho statistical modeling.
- **ML/AI:** TensorFlow, Scikit-learn cho predictive analytics.

- **Visualization:** D3.js, ggplot2, BI platforms.
- **Real-time Processing:** Apache Kafka, Flink cho streaming DSS.

1.2.5 Ứng dụng và lợi ích

DSS được áp dụng rộng rãi trong tài chính (risk assessment), y tế (diagnosis support), logistics (route optimization), và giao thông (congestion prediction). Lợi ích: cải thiện chất lượng quyết định, giảm thời gian xử lý, tăng tính minh bạch, hỗ trợ scenario planning.

2 Tổng quan vấn đề và mô tả bài toán

2.1 Tổng quan về vấn đề tắc nghẽn giao thông

Tắc nghẽn giao thông là vấn đề nghiêm trọng trong quá trình đô thị hóa, ảnh hưởng đến kinh tế, môi trường và chất lượng cuộc sống của người dân. Theo INRIX Global Traffic Scorecard (2024–2025), ùn tắc gây thiệt hại hàng trăm tỷ USD mỗi năm, làm tăng lượng khí thải nhà kính và kéo dài thời gian di chuyển tại các đô thị lớn.

Trong bối cảnh này, việc ứng dụng Kho dữ liệu (Data Warehouse) kết hợp với Hệ hỗ trợ quyết định (Decision Support System – DSS) vào giao thông đô thị trở nên cần thiết. Công nghệ này giúp thu thập, phân tích và dự báo tình trạng tắc nghẽn hiệu quả, từ đó hỗ trợ cơ quan quản lý ra quyết định điều tiết phương tiện và quy hoạch hạ tầng giao thông thông minh, hướng đến xây dựng hệ thống giao thông thông minh (ITS).

2.2 Mục tiêu của dự án

Mục tiêu của dự án là xây dựng kho dữ liệu giao thông và hệ hỗ trợ ra quyết định (DSS) nhằm phân tích và dự báo tình trạng tắc nghẽn giao thông tại các khu vực đô thị lớn. Thông qua việc ứng dụng các công nghệ Kho dữ liệu (Data Warehouse), dự án hướng đến việc tích hợp, lưu trữ, xử lý và phân tích dữ liệu giao thông từ nhiều nguồn khác nhau để:

- Phát hiện xu hướng tắc nghẽn theo thời gian và không gian, như khu vực, thời điểm hoặc điều kiện thời tiết ảnh hưởng đến mật độ giao thông.
- Xây dựng mô hình dự báo tắc nghẽn giao thông, giúp đưa ra cảnh báo sớm và đề xuất giải pháp điều phối giao thông hiệu quả.
- Hỗ trợ ra quyết định cho các cơ quan quản lý giao thông, thông qua hệ thống báo cáo và trực quan hóa dữ liệu, giúp hoạch định chính sách hoặc tối ưu hạ tầng.

Như vậy, dự án không chỉ tập trung vào việc quản lý dữ liệu mà còn nhằm chuyển hóa dữ liệu giao thông lớn (big data) thành tri thức có ích cho việc quy hoạch, điều hành và dự báo tình trạng giao thông trong tương lai.

2.3 Vai trò của Kho dữ liệu (DW) và hệ hỗ trợ ra quyết định (DSS) trong bài toán giao thông

Kho dữ liệu (DW) đóng vai trò là nền tảng lưu trữ tập trung, tích hợp dữ liệu từ nhiều nguồn khác nhau: cảm biến giao thông, GPS, camera giám sát, và dữ liệu thời tiết. Dữ liệu trong DW được chuẩn hóa và lưu trữ phục vụ cho phân tích đa chiều (OLAP). Hệ hỗ trợ quyết định (DSS) được xây dựng trên nền kho dữ liệu, cho phép nhà quản lý:

- Truy vấn, phân tích xu hướng tắc nghẽn giao thông.
- Hiển thị thông tin trực quan qua dashboard (bảng điều khiển).
- Đưa ra các khuyến nghị hoặc dự báo tắc nghẽn trong tương lai gần.

Sự kết hợp giữa DW + DSS giúp chuyển dữ liệu thô thành thông tin có giá trị phục vụ ra quyết định, góp phần xây dựng hệ thống Giao thông Thông minh (ITS – Intelligent Transportation System).

2.4 Nguồn dữ liệu

2.4.1 Tổng quan nguồn dữ liệu

Dự án sử dụng tập dữ liệu "US Traffic Congestions (2016–2022)"do tác giả Sobhan Moosavi công bố trên Kaggle. Link dữ liệu: US Traffic Congestions Dataset
Đây là tập dữ liệu quốc gia về tình trạng tắc nghẽn giao thông tại 49 bang Hoa Kỳ, được thu thập từ tháng 2/2016 đến tháng 9/2022 qua nhiều API thu thập dữ liệu giao thông thời gian thực. Các API này tổng hợp thông tin từ Bộ Giao thông Hoa Kỳ (US DOT), sở giao thông các bang, camera giao thông, cảm biến đường bộ và cơ quan thực thi pháp luật.

Tập dữ liệu bao gồm khoảng 33 triệu bản ghi sự kiện tắc nghẽn, kèm phiên bản mẫu 2 triệu sự kiện để thuận tiện xử lý và phân tích. Dữ liệu chứa các thông tin quan trọng như thời gian, vị trí địa lý, mức độ tắc nghẽn, điều kiện thời tiết, tốc độ lưu thông và độ trễ giao thông - là cơ sở thực tiễn để xây dựng kho dữ liệu phục vụ phân tích,

thống kê và dự báo tắc nghẽn giao thông.

Mục đích sử dụng:

- Phân tích xu hướng tắc nghẽn theo không gian và thời gian.
- Tìm hiểu mối tương quan giữa tắc nghẽn và điều kiện thời tiết.
- Xây dựng kho dữ liệu hỗ trợ quyết định cho hệ thống giao thông thông minh.

2.4.2 Cấu trúc dữ liệu

Tập dữ liệu US Traffic Congestions (2016–2022) được tổ chức một cách chi tiết và toàn diện, bao gồm 30 trường dữ liệu chính. Các trường này được phân loại rõ ràng thành các nhóm thông tin về sự kiện tắc nghẽn, vị trí địa lý, thời gian, và điều kiện khí tượng, tạo thành nền tảng dữ liệu đa chiều cho quá trình phân tích và xây dựng mô hình dự báo. Cấu trúc chi tiết của các cột được liệt kê trong bảng dưới đây:

STT	Tên cột	Mô tả nội dung
1	ID	Mã định danh duy nhất cho mỗi sự kiện tắc nghẽn giao thông.
2	Severity	Mức độ nghiêm trọng của sự kiện, thể hiện trên thang đo số (ví dụ 1–4).
3	Start_Lat	Vĩ độ điểm bắt đầu của đoạn đường xảy ra tắc nghẽn.
4	Start_Lng	Kinh độ điểm bắt đầu của đoạn đường xảy ra tắc nghẽn.
5	StartTime	Thời gian bắt đầu của sự kiện tắc nghẽn.
6	EndTime	Thời gian kết thúc của sự kiện.
7	Distance(mi)	Độ dài đoạn đường bị ảnh hưởng (đơn vị: dặm).
8	DelayFromTypicalTraffic(mins)	Độ trễ so với lưu lượng giao thông thông thường (phút).
9	DelayFromFreeFlowSpeed(mins)	Độ trễ so với tốc độ lưu thông tự do (phút).
10	Congestion_Speed	Tốc độ trung bình của phương tiện khi xảy ra tắc nghẽn.
11	Description	Mô tả chi tiết về sự kiện (dưới dạng văn bản).
12	Street	Tên đường hoặc tuyến phố nơi xảy ra sự kiện.
13	City	Thành phố nơi sự kiện diễn ra.
14	County	Quận/hạt quản lý khu vực xảy ra sự kiện.
15	State	Bang nơi sự kiện diễn ra.

STT	Tên cột	Mô tả nội dung
16	Country	Quốc gia (thường là "US").
17	ZipCode	Mã bưu chính (ZIP code) của khu vực, có thể bị thiếu ở một số bản ghi.
18	LocalTimeZone	Múi giờ địa phương nơi xảy ra sự kiện (ví dụ: America/New_York).
19	WeatherStation_AirportCode	Mã sân bay hoặc trạm thời tiết gần nhất cung cấp dữ liệu khí tượng.
20	WeatherTimeStamp	Thời điểm ghi nhận dữ liệu thời tiết.
21	Temperature(F)	Nhiệt độ không khí (đơn vị: độ F).
22	WindChill(F)	Chỉ số nhiệt độ cảm nhận do gió (Wind chill, °F).
23	Humidity(%)	Độ ẩm tương đối trong không khí (%).
24	Pressure(in)	Áp suất khí quyển (đơn vị: inch Hg).
25	Visibility(mi)	Tầm nhìn của người lái xe tại thời điểm xảy ra sự kiện (đơn vị: dặm).
26	WindDir	Hướng gió (ký hiệu chữ hoặc độ).
27	WindSpeed(mph)	Tốc độ gió (đơn vị: dặm/giờ).
28	Precipitation(in)	Lượng mưa hoặc tuyết rơi (inch).
29	Weather_Event	Loại hiện tượng thời tiết (mưa, tuyết, sấm sét, sương mù,...).
30	Weather_Conditions	Mô tả chi tiết điều kiện thời tiết tại thời điểm ghi nhận.

Bảng 2: Cấu trúc bộ dữ liệu US Traffic Congestions (2016–2022)

2.4.3 Đặc điểm dữ liệu

- **Kích thước lớn:** Dữ liệu bao gồm hàng triệu bản ghi sau khi mở rộng, giúp đảm bảo tính đại diện cao.
- **Tích hợp đa nguồn:** Kết hợp giữa dữ liệu giao thông, địa lý và thời tiết.
- **Dữ liệu thời gian–không gian:** Cho phép phân tích xu hướng tắc nghẽn theo khu vực, thành phố, thời điểm hoặc điều kiện thời tiết.

- **Thiếu dữ liệu (missing values):** Một số cột thời tiết (`WindChill`, `Precipitation`, `WindSpeed`) có thể trống ở nhiều bản ghi.
- **Độ chính xác địa lý:** Các cột `Start_Lat`, `Start_Lng` cho phép định vị bản đồ (GIS) và phân tích tắc nghẽn theo tọa độ.
- **Tính nhất quán:** Cần chuẩn hóa giá trị chữ (`City`, `Weather_Event`, `Weather_Conditions`) để tránh trùng tên do khác kiểu viết.

2.4.4 Phân tích tổng quan nguồn dữ liệu

Nguồn dữ liệu cho thấy mối tương quan chặt chẽ giữa tình trạng tắc nghẽn giao thông và các yếu tố môi trường. Dữ liệu này mở ra nhiều hướng phân tích tiềm năng, giúp hiểu rõ hơn về đặc trưng và nguyên nhân của hiện tượng tắc nghẽn. Một số hướng khai thác đáng chú ý bao gồm:

- Phân tích ảnh hưởng của điều kiện thời tiết (mưa, tuyết, sương mù) đến mức độ và thời gian xảy ra tắc nghẽn.
- Đánh giá phân bố không gian của các sự kiện tắc nghẽn theo bang, thành phố hoặc khu vực có mật độ giao thông cao, nhằm xác định các "điểm nóng" về giao thông.
- Phân tích xu hướng theo thời gian, như sự biến động của tắc nghẽn trong giờ cao điểm, cuối tuần hoặc các dịp lễ hội.
- Khảo sát mối quan hệ giữa tốc độ trung bình (Congestion_Speed) và độ trễ (DelayFromTypicalTraffic) để đánh giá hiệu suất lưu thông và mức độ ảnh hưởng của ùn tắc.

Những kết quả phân tích này có thể đóng vai trò là nền tảng dữ liệu cho việc xây dựng Kho dữ liệu giao thông (Traffic Data Warehouse), hệ thống hỗ trợ ra quyết định (Decision Support System – DSS), hoặc mô hình dự báo tắc nghẽn giao thông trong tương lai.

3 Thu thập và Tiền xử lý dữ liệu

3.1 Khái niệm tiền xử lí dữ liệu

Tiền xử lý dữ liệu (Data Preprocessing) là quá trình áp dụng các kỹ thuật nhằm biến đổi dữ liệu thô thành dữ liệu sạch, có tổ chức và sẵn sàng cho các bước phân tích tiếp theo. Trong quá trình xây dựng kho dữ liệu, tiền xử lý giúp đảm bảo dữ liệu thu thập từ nhiều nguồn khác nhau được chuẩn hóa, làm sạch và tích hợp một cách đồng nhất, tạo cơ sở vững chắc cho việc phân tích và hỗ trợ ra quyết định chính xác.
Quy trình tiền xử lý thường bao gồm các bước chính: làm sạch dữ liệu (Data Cleaning), tích hợp dữ liệu (Data Integration), chuyển đổi dữ liệu (Data Transformation) và giảm kích thước dữ liệu (Data Reduction).
Trước khi tiến hành phân tích và xây dựng mô hình dự đoán, dữ liệu cần được tiền xử lý để đảm bảo chất lượng và tính nhất quán.

3.2 Phân tích cột

Trước khi đi vào xử lý tiền xử lý, ta phải xem các cột sẵn có của tập dữ liệu. Tập dữ liệu có các cột như sau:

- **id:** Mã định danh duy nhất cho mỗi bản ghi.
- **severity:** Mức độ nghiêm trọng của sự kiện/tai nạn giao thông
- **start_lat, start_lng:** Vĩ độ và kinh độ của điểm bắt đầu sự kiện.
- **starttime, endtime:** Thời gian bắt đầu và kết thúc sự kiện.
- **distance(mi):** Khoảng cách của đoạn đường bị ảnh hưởng (miles).
- **delayfromtypicaltraffic(mins):** Thời gian trễ so với lưu lượng giao thông điển hình.
- **delayfromfreeflowspeed(mins):** Thời gian trễ so với tốc độ lưu thông tự do.

- **congestion_speed:** Tốc độ trung bình trong đoạn đường bị tắc nghẽn.
- **description:** Mô tả chi tiết sự kiện.
- **street, city, county, state, country, zipcode:** Thông tin vị trí chi tiết của sự kiện.
- **localtimezone:** Múi giờ địa phương.
- **weatherstation_airportcode:** Mã sân bay/ trạm thời tiết gần nhất.
- **weathertimestamp:** Thời điểm ghi nhận dữ liệu thời tiết.
- **temperature(f), windchill(f):** Nhiệt độ và cảm giác lạnh.
- **humidity(%):** Độ ẩm phần trăm.
- **pressure(in):** Áp suất khí quyển.
- **visibility(mi):** Tầm nhìn.
- **winddir:** Hướng gió.
- **windspeed(mph):** Tốc độ gió.
- **precipitation(in):** Lượng mưa.
- **weather_event:** Loại hiện tượng thời tiết (mưa, tuyết, sương mù...).
- **weather_conditions:** Điều kiện thời tiết.

3.3 Xử lý giá trị thiếu (N/A)

Bảng 3 cho thấy phần trăm giá trị thiếu của từng cột trong tập dữ liệu.

Bảng 3: Tỷ lệ giá trị thiếu của các cột dữ liệu

Cột	% Giá trị thiếu
weather_event	93.60%
precipitation(in)	41.26%
windchill(f)	37.96%
windspeed(mph)	9.47%
visibility(mi)	2.92%
weather_conditions	2.85%
winddir	2.37%
humidity(%)	2.31%
temperature(f)	2.20%
pressure(in)	2.06%
weathertimestamp	1.61%
weatherstation_airportcode	0.17%
starttime	0.09%
endtime	0.09%
localtimezone	0.09%
zipcode	0.07%
city	0.01%
street	0.01%

weather_event

Đối với cột **weather_event**, các giá trị được lưu ở dạng chuỗi **"event_1;event_2;..."**. Để thuận tiện cho việc xử lý và phân tích, ta tạo thêm cột mới **weather_event_list** và chuyển các giá trị trong cột **weather_event** thành danh sách (list) các hiện tượng thời tiết.

Đoạn code thực hiện điều này:

```
df['weather_event_list'] = df['weather_event'].apply(
    lambda x: x.split(';') if pd.notnull(x) else []
)
```

precipitation(in)

Đối với cột **precipitation**, quá trình xử lý được thực hiện theo hai bước:

1. Nếu **weather_event** là **None** và giá trị **precipitation** bị thiếu, gán giá trị bằng **0** (tức không có mưa).

2. Với các bản ghi còn thiếu khác, thay thế giá trị **precipitation** bằng **trung vị lượng mưa** của từng nhóm **weather_event**.

Sau khi áp dụng hai bước trên, các bản ghi vẫn còn giá trị **precipitation** bị thiếu sẽ được loại bỏ khỏi tập dữ liệu.

Các cột số

Đối với các cột số **windspeed(mph)**, **visibility(mi)**, **humidity(%)**, **temperature(f)**, và **pressure(in)**, các giá trị thiếu được thay thế bằng **trung vị (median)** của từng cột. Việc sử dụng trung vị giúp giảm ảnh hưởng của các giá trị ngoại lai so với việc dùng trung bình.

Cột city, street

Ta loại bỏ các hàng có giá trị n/a tại 2 cột này, vì giá trị thiếu của nó chỉ tồn tại với 0.01%.

Cột weathertimestamp và weather_conditions

Đối với hai cột thông tin thời tiết **weathertimestamp** và **weather_conditions**, các giá trị thiếu được điền dựa trên vị trí địa lý (**city**, **street**) và thời gian xảy ra sự kiện.

Cột weathertimestamp và weather_conditions

- Dữ liệu được sắp xếp theo **city**, **street**, **starttime**.
- Sử dụng phương pháp **forward fill** và **backward fill** trong từng nhóm **(city, street)** để tận dụng các giá trị thời tiết gần nhất về không gian và thời gian.
- Nếu **weathertimestamp** vẫn còn thiếu, giá trị được điền bằng $\textbf{starttime} + \frac{\textbf{endtime} - \textbf{starttime}}{2}$, tức là thời điểm giữa sự kiện.
- Nếu **weather_conditions** vẫn còn thiếu, sử dụng **mode** (giá trị xuất hiện nhiều nhất) của toàn bộ cột làm giá trị thay thế cuối cùng.

Các cột không ảnh hưởng

Một số cột không đóng góp đáng kể thông qua suy luận cho quá trình phân tích và mô hình hóa được loại bỏ để giảm nhiễu dữ liệu và kích thước tập dữ liệu. Cụ thể, ta loại bỏ các cột: **winddir**, **weatherstation_airportcode**, và **windchill(f)**.

3.4 Chuẩn hóa dữ liệu

Dữ liệu thời gian

Các cột thời gian **starttime**, **endtime** và **weathertimestamp** được chuyển sang kiểu **datetime** của pandas để thuận tiện cho việc so sánh, tính toán và trích xuất các đặc trưng thời gian (giờ, ngày, tháng...).

Dữ liệu số

Để đảm bảo các đặc trưng số có cùng thang đo và giúp mô hình học hiệu quả hơn, ta chuẩn hoá các cột số về phân phối chuẩn (mean = 0, std = 1) bằng **StandardScaler** của thư viện `scikit-learn`.
Các cột được chuẩn hoá:

- **severity**
- **start_lat**

- **start_lng**
- **distance(mi)**
- **delayfromtypicaltraffic(mins)**
- **delayfromfreeflowspeed(mins)**
- **temperature(f)**
- **humidity(%)**
- **pressure(in)**
- **visibility(mi)**
- **windspeed(mph)**
- **precipitation(in)**

Dữ liệu phân loại

- **weather_event_list**:

Để chuyển các cột dạng phân loại (categorical) sang dạng số, các phương pháp mã hoá sau được áp dụng:

- **Mã hoá đa nhãn (Multi-label Encoding)** Cột **weather_event_list** chứa nhiều giá trị hiện tượng thời tiết cho mỗi bản ghi. Sử dụng **MultiLabelBinarizer** để biến mỗi loại thời tiết thành một cột nhị phân (0/1).
- **Mã hoá đơn nhãn (Label Encoding)** Với các cột phân loại thông thường, ta sử dụng **LabelEncoder** để chuyển từng giá trị chuỗi sang số nguyên.

3.5 Thu thập dữ liệu vào Staging

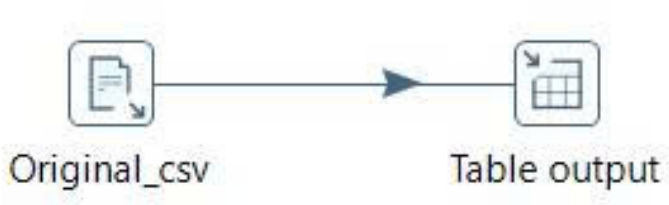
Sau khi có được nguồn dữ liệu, tiến hành lưu trữ dữ liệu về khí tượng và nồng độ chất trong không khí vào Staging với PostgreSQL. Các bảng lưu trữ trong PostgreSQL được tạo như sau:

```
DROP TABLE IF EXISTS staging_congestion_raw;

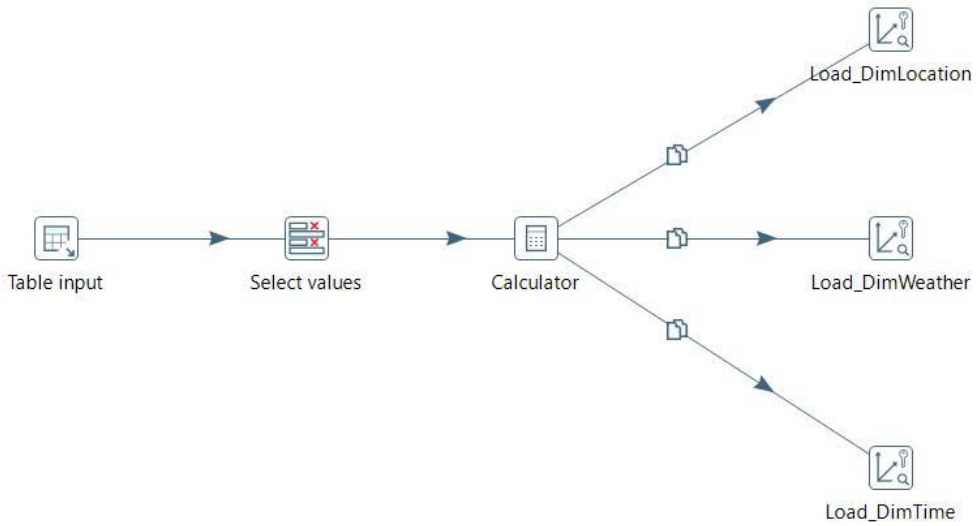
CREATE TABLE staging_congestion_raw (
    id TEXT,
    severity TEXT,
    start_lat TEXT,
    start_lng TEXT,
    starttime TEXT,
    endtime TEXT,
    distance_mi TEXT,                    -- Gốc: Distance(mi)
    delay_typical_traffic_mins TEXT,     -- Gốc: DelayFromTypicalTraffic(mins)
    delay_free_flow_speed_mins TEXT,     -- Gốc: DelayFromFreeFlowSpeed(mins)
    congestion_speed TEXT,
    description TEXT,
    street TEXT,
    city TEXT,
    county TEXT,
    state TEXT,
    country TEXT,
    zipcode TEXT,
    local_timezone TEXT,
    weather_station_code TEXT,           -- Gốc: WeatherStation_AirportCode
    weather_timestamp TEXT,
    temperature_f TEXT,                  -- Gốc: Temperature(F)
    wind_chill_f TEXT,                   -- Gốc: WindChill(F)
    humidity_percent TEXT,               -- Gốc: Humidity(%)
    pressure_in TEXT,                    -- Gốc: Pressure(in)
    visibility_mi TEXT,                  -- Gốc: Visibility(mi)
    wind_dir TEXT,
    wind_speed_mph TEXT,                 -- Gốc: WindSpeed(mph)
    precipitation_in TEXT,               -- Gốc: Precipitation(in)
```

```
32     weather_event TEXT,
33     weather_conditions TEXT,
34
35     loaded_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
36 );
```

Việc lưu trữ dữ liệu vào Staging được tiến hành với Pentaho. Quá trình ETL từ nguồn dữ liệu vào Staging được thực hiện như sau:



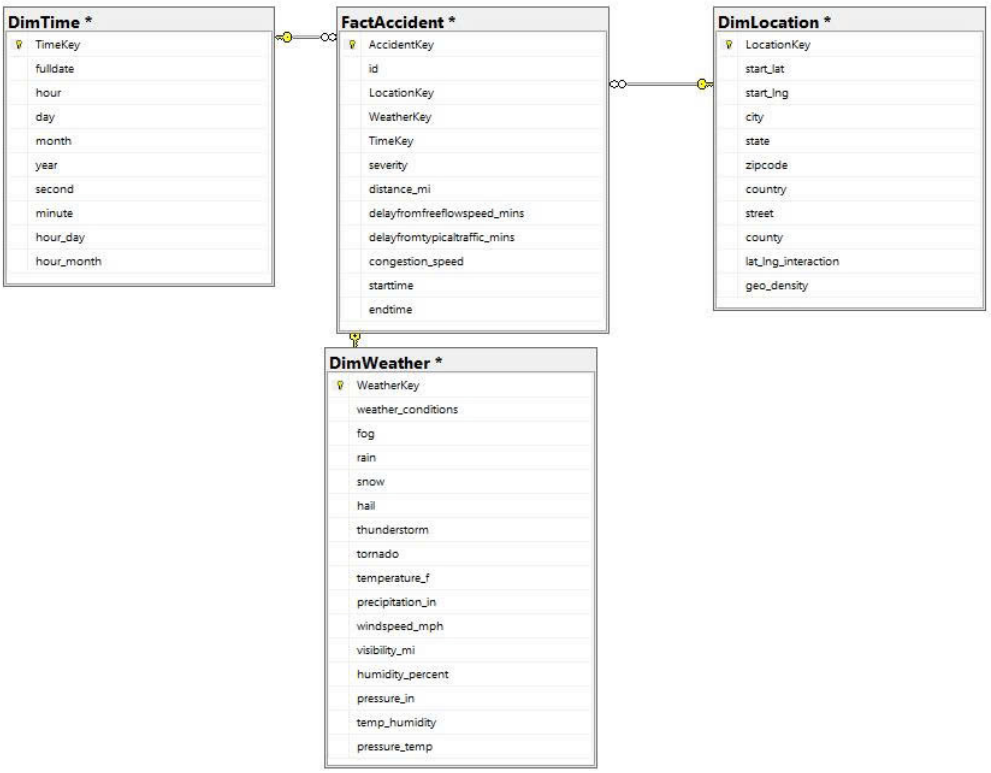
Hình 1: Nạp dữ liệu cho stagging



Hình 2: Tính toán và nạp dữ liệu vào Dim table

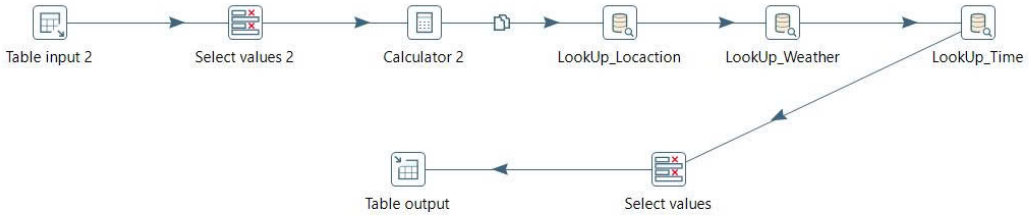
3.6 Xây dựng kho dữ liệu

Từ dữ liệu Staging, tiến hành đổ dữ liệu vào Microsoft SQL Server định kì theo tháng để xây dựng kho dữ liệu. Đầu tiên, thiết kế kho dữ liệu dưới dạng Star:



Hình 3: Star Schema

Ở dạng lược đồ ngôi sao, kho dữ liệu sẽ có một bảng Fact (FactAccident - lưu dữ liệu giá của các thông số) và ba bảng Dimension (dimTime - lưu các thời điểm đo, dimLocation - lưu vị trí đo và dimWeather - lưu sự kiền thời tiết). Tuy nhiên, ở bảng dim_datetime, thuộc tính season thường xuyên lặp lại và cần thêm những thông tin khác như tháng bắt đầu và tháng kết thúc của mùa, do đó, thêm một bảng dim_season để không chỉ phân tích theo từng ngày riêng biệt mà có thể phân tích dữ liệu theo mùa. Sau khi đã xây dựng kho dữ liệu và có dữ liệu trong Staging, sử dụng Pentaho để đổ dữ liệu từ Staging vào Warehouse.



Hình 4: Dữ liệu từ Staging vào Warehouse

4 Phương pháp nghiên cứu và mô hình hoá

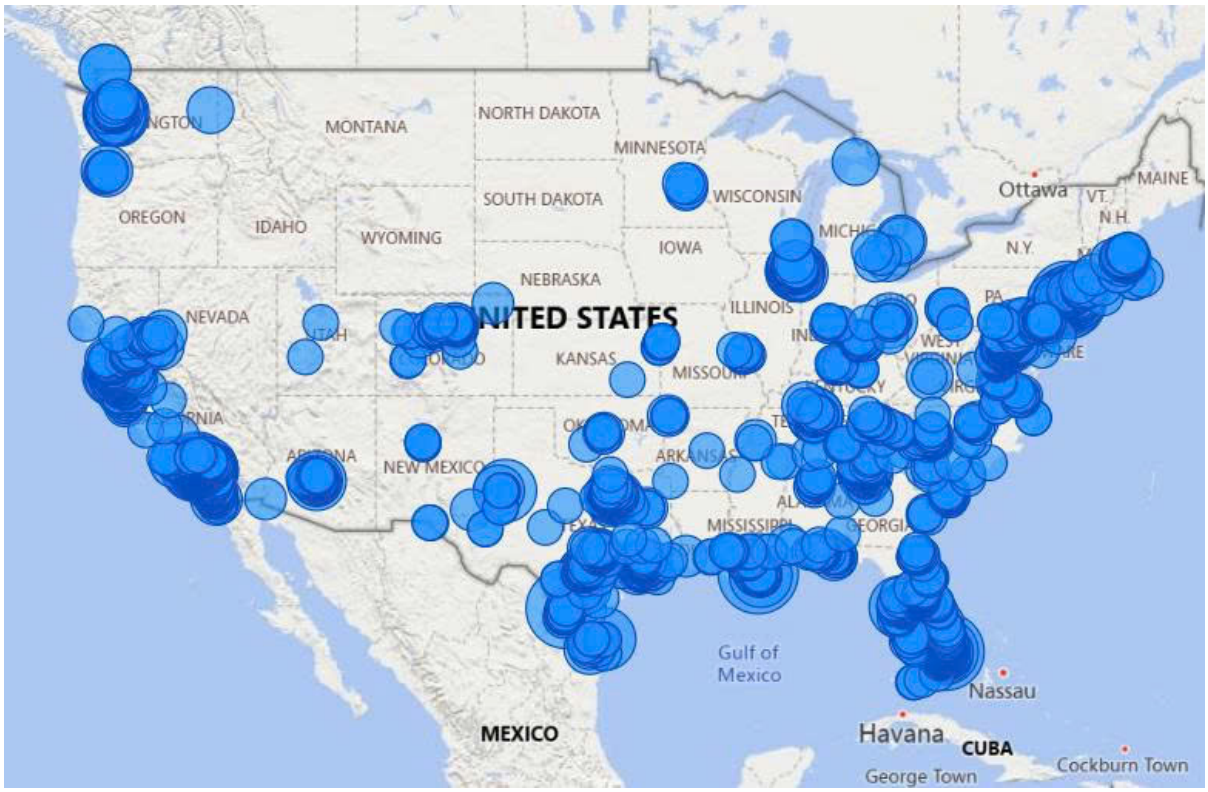
Để giải quyết bài toán dự báo tắc nghẽn giao thông, nhóm thực hiện theo quy trình khép kín gồm hai phương pháp tiếp cận bổ trợ cho nhau:

1. **Phương pháp tiếp cận Khám phá (Exploratory Approach):** Sử dụng Phân tích trực quan (Visualization) và Thống kê mô tả để hiểu cấu trúc dữ liệu và xác định tính chất phi tuyến của bài toán.

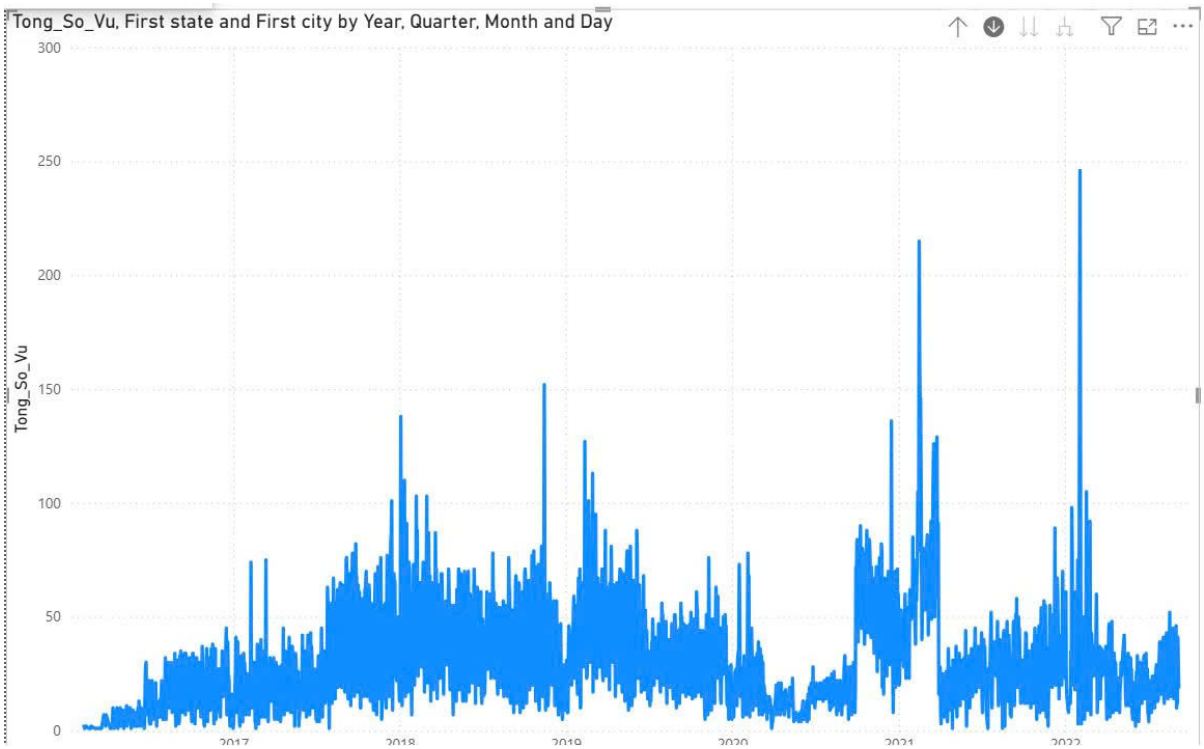
2. **Phương pháp tiếp cận Mô hình hóa (Modeling Approach):** Sử dụng Kỹ thuật Khai phá dữ liệu (Data Mining) với hai thuật toán đại diện là XGBoost (Ensemble Learning) và MLP (Deep Learning) để xây dựng bộ dự báo.

4.1 Phương pháp 1: Tiếp cận khám phá và phân tích trực quan

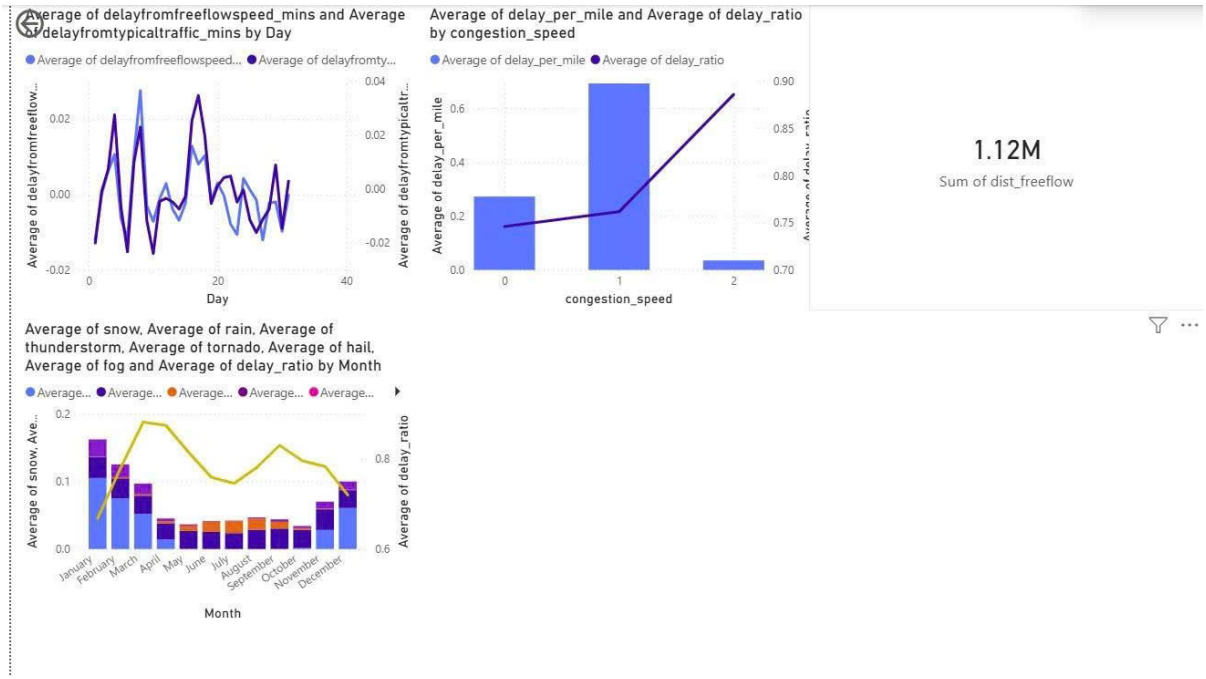
Trước khi xây dựng mô hình, việc hiểu rõ mối quan hệ giữa các biến đầu vào (thời tiết, địa lý) và biến mục tiêu (*Congestion Speed*) là bước tiên quyết để lựa chọn thuật toán phù hợp.



Hình 5: Phân bố mật độ tắc nghẽn



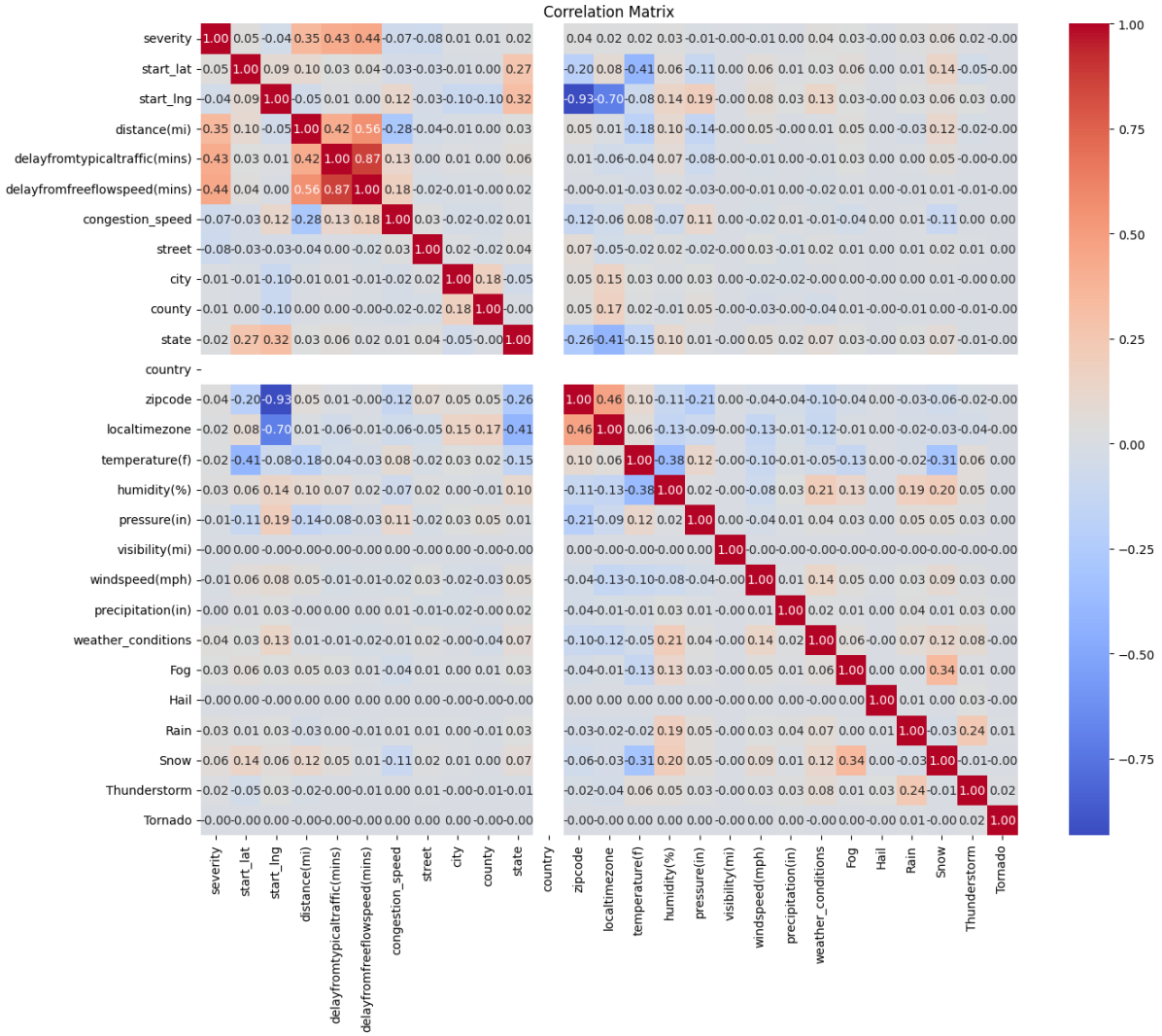
Hình 6: Mật độ tắt nghẽn qua các năm của một thành phố



Hình 7: Các ảnh hưởng khác với tốc độ tắt nghẽn

4.1.1 Ma trận tương quan (Correlation Analysis)

Nhóm xây dựng ma trận tương quan (Correlation Matrix) để kiểm tra mức độ phụ thuộc tuyến tính giữa các biến số.



Hình 8: Ma trận tương quan giữa các đặc trưng và biến mục tiêu

Phân tích kết quả

- Quan sát ma trận (Hình 8), hệ số tương quan giữa *Congestion Speed* và các đặc trưng đơn lẻ như *Temperature*, `Precipitation`, *Humidity* đều rất thấp (xấp xỉ 0).

- **Kết luận phương pháp:** Điều này chứng minh rằng tắc nghẽn giao thông không có mối quan hệ tuyến tính đơn giản với các yếu tố môi trường. Do đó, các mô hình hồi quy tuyến tính (Linear Regression) sẽ không hiệu quả. Bài toán đòi hỏi các mô hình có khả năng học các mối quan hệ phi tuyến phức tạp (Non-linear relationships).

4.1.2 Phân tích phân phối (Distribution Analysis)

Để hiểu rõ hơn về tính chất dữ liệu sau khi tiền xử lý và chuẩn hóa, nhóm thực hiện vẽ biểu đồ tần suất (Histogram) và biểu đồ phân tán (Scatter Plot) cho các đặc trưng.



Hình 9: Tất cả các biểu đồ đặc trưng sau tiền xử lý.

Quan sát chính

1. **Biểu đồ tần suất (Cột trái):**

   - Hầu hết các biến số (ví dụ: thành phố, quốc gia, khoảng cách, độ ẩm, áp suất, mức độ nghiêm trọng) có phân phối lệch hoặc có đỉnh rõ rệt, với một số có đỉnh duy nhất (ví dụ: mức độ nghiêm trọng, giờ bắt đầu) và số khác có nhiều đỉnh hoặc phân bố đồng đều hơn (ví dụ: điều kiện thời tiết).

   - Giá trị chuẩn hóa thường nằm trong khoảng từ $[-2, 2]$, với tần suất thay đổi

từ vài nghìn đến hơn 10.000 trong một số trường hợp (ví dụ: mức độ nghiêm trọng).

- Các biến như giờ bắt đầu và nhiệt độ cho thấy phân phối đa mô hình, ám chỉ sự tập trung quanh các giá trị cụ thể.

2. **Biểu đồ phân tán (Cột phải):**

- Các biểu đồ phân tán thường cho thấy không có mối tương quan tuyến tính mạnh giữa các biến số chuẩn hóa và tốc độ ùn tắc, vốn dao động từ 0 đến 2.
- Một số biến (ví dụ: thành phố, quốc gia, khoảng cách) cho thấy phân bố phẳng hoặc gần phẳng của tốc độ ùn tắc trên phạm vi của chúng, cho thấy không có mối quan hệ rõ rệt.
- Một số khác (ví dụ: mức độ nghiêm trọng, độ ẩm) cho thấy sự biến thiên hơn trong tốc độ ùn tắc, với một số tập trung (ví dụ: quanh $1.0 - 1.5$ cho mức độ nghiêm trọng), nhưng vẫn không có xu hướng rõ ràng.
- Một số điểm dữ liệu ngoại lai hoặc thưa thớt có thể thấy trong một số đồ thị (ví dụ: sấm sét so với tốc độ ùn tắc), ám chỉ các điều kiện hiếm gặp ảnh hưởng đến tốc độ.

Nhận xét

- Việc chuẩn hóa các biến cho phép so sánh trên các thang đo khác nhau, nhưng các biểu đồ tần suất cho thấy một số biến tập trung hơn (ví dụ: mức độ nghiêm trọng quanh 0) trong khi số khác phân bố đồng đều hơn (ví dụ: điều kiện thời tiết).
- Các biểu đồ phân tán cho thấy tốc độ ùn tắc bị ảnh hưởng bởi nhiều yếu tố, nhưng không biến số nào thống trị mối quan hệ, chỉ ra sự phụ thuộc phức tạp, có thể là đa biến.

4.2 Phương pháp 2: Tiếp cận khai phá dữ liệu và dự báo

4.2.1 Cơ sở lý thuyết

Dựa trên kết luận từ phương pháp khám phá (dữ liệu phi tuyến và phức tạp), nhóm lựa chọn hai thuật toán học máy mạnh mẽ để giải quyết bài toán phân loại mức độ tắc nghẽn (Classification).

4.2.1.1 Thuật toán 1: XGBoost (Extreme Gradient Boosting)

XGBoost (Extreme Gradient Boosting) là một thuật toán học máy mạnh mẽ thuộc nhóm ensemble learning, cụ thể là phương pháp boosting. Nó được thiết kế để tối ưu hiệu năng và tốc độ khi huấn luyện mô hình cây quyết định. Đây là đại diện tiêu biểu cho nhóm phương pháp **Ensemble Learning** (Học kết hợp), cụ thể là kỹ thuật Boosting.

Nguyên lý hoạt động:

XGBoost xây dựng mô hình bằng cách kết hợp nhiều cây quyết định theo từng vòng lặp. Mỗi cây mới được huấn luyện nhằm sửa lỗi của cây trước đó, giúp mô hình học tốt hơn theo thời gian.

1. Hàm mục tiêu

Ở vòng boosting thứ t, mô hình XGBoost tối ưu hàm mục tiêu sau:

$$\mathcal{L}^{(t)} = \sum_{i=1}^{n} l(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) + \Omega(f_t)$$

Trong đó:

- l: hàm mất mát (thường dùng MSE: $l(y, \hat{y}) = (y - \hat{y})^2$)
- $\hat{y}_i{}^{(t-1)}$: dự đoán của mô hình tại bước t - 1
- f_t: cây quyết định mới tại bước t
- $\Omega(f_t)$: thành phần regularization giúp chống overfitting

$$\Omega(f)f_t = \gamma T + \frac{1}{2}\lambda \sum_{j=1}^{T} w_j^2$$

Trong đó:

- T: số lá của cây
- w_j: trọng số lá thứ j
- γ, λ: hệ số điều chỉnh độ phức tạp mô hình

2. Xấp xỉ Taylor bậc hai

XGBoost sử dụng xấp xỉ Taylor bậc hai để làm trơn hàm mất mát:

$$\mathcal{L}^{(t)} \approx \sum_{i=1}^{n} \left[g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i) \right] + \Omega(f_t)$$

Trong đó: - $g_i = \frac{\partial l(y_i, \hat{y}_i^{(t-1)})}{\partial \hat{y}_i^{(t-1)}}$ (đạo hàm bậc 1) - $h_i = \frac{\partial^2 l(y_i, \hat{y}_i^{(t-1)})}{\partial (\hat{y}_i^{(t-1)})^2}$ (đạo hàm bậc 2)

3. Tính giá trị tối ưu tại mỗi lá

Với mỗi lá j, tập mẫu thuộc lá là I_j. Tổng gradient và hessian tại lá:

$$G_j = \sum_{i \in I_j} g_i, \quad H_j = \sum_{i \in I_j} h_i$$

Giá trị đầu ra tối ưu tại lá:

$$w_j^* = -\frac{G_j}{H_j + \lambda}$$

Giá trị hàm mục tiêu tại điểm cực tiểu:

$$\mathcal{L}^{(t)} = -\frac{1}{2} \sum_{j=1}^{T} \frac{G_j^2}{H_j + \lambda} + \gamma T$$

Quá trình huấn luyện:

Quá trình huấn luyện XGBoost bao gồm các bước chính sau:

1. **Khởi tạo mô hình**: Dự đoán ban đầu được thiết lập là xác suất trung bình của các lớp trong tập huấn luyện.

2. **Tính toán gradient và hessian:** Các gradient (g_i) và hessian (h_i) được tính cho mỗi mẫu dựa trên hàm mất mát phân loại.

3. **Xây dựng cây quyết định:** Một cây mới được tạo ra nhằm tối ưu hóa hàm mất mát, sử dụng thông tin từ gradient và hessian để xác định cách chia nhánh tốt nhất.

4. **Cập nhật dự đoán:** Dự đoán của mô hình được cập nhật bằng cách cộng thêm giá trị từ cây mới.

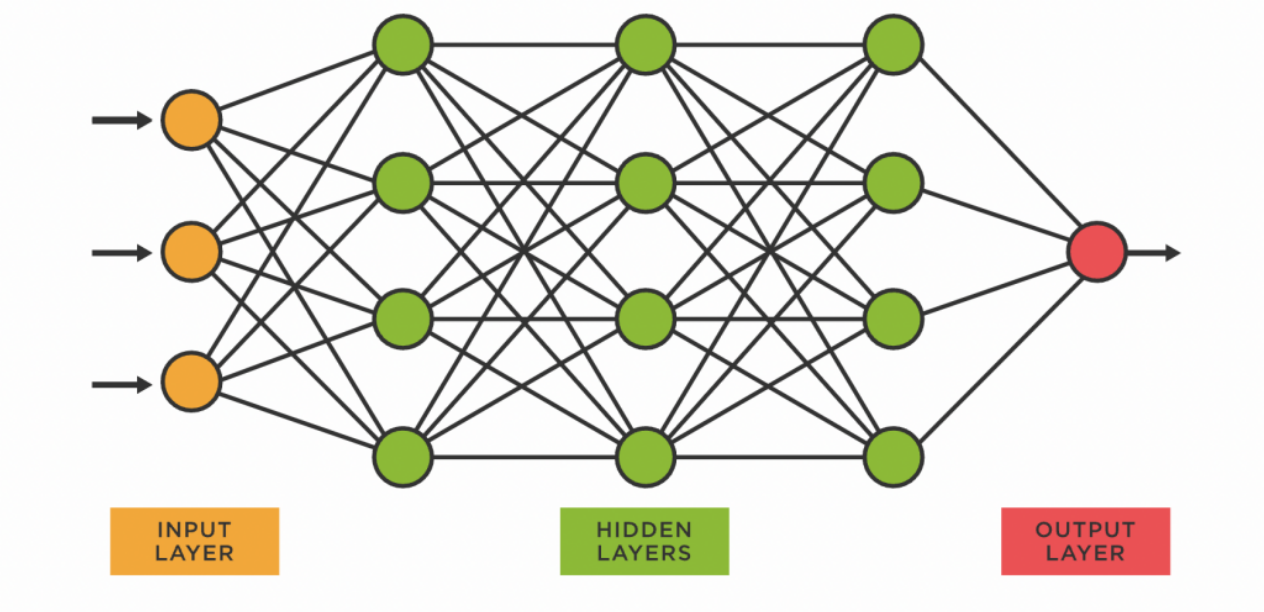
5. **Regularization:** Thành phần regularization giúp kiểm soát độ phức tạp của mô hình và ngăn ngừa overfitting.

6. **Lặp lại quá trình:** Các bước trên được lặp lại cho đến khi đạt số vòng lặp tối đa hoặc không còn cải thiện đáng kể.

XGBoost là một lựa chọn mạnh mẽ cho bài toán phân loại, đặc biệt khi dữ liệu:
Không cân bằng: Mô hình cho phép điều chỉnh trọng số lớp thiểu số thông qua tham số scale_pos_weight, giúp cải thiện độ chính xác và F1-score.
Có mối quan hệ phi tuyến yếu: Nhờ khả năng xây dựng cây quyết định lặp lại và tối ưu hóa liên tục, XGBoost có thể học được các mẫu phức tạp mà các mô hình tuyến tính không thể nắm bắt.
Kỹ thuật boosting giúp mô hình học từ các sai sót của những cây trước đó, từ đó cải thiện hiệu suất phân loại qua từng vòng lặp. Đây là lý do XGBoost thường đạt kết quả tốt trong các bài toán phân loại thực tế, đặc biệt khi dữ liệu có độ phức tạp cao và phân bố không đều giữa các lớp.

4.2.1.2 Thuật toán 2: Mạng nơ-ron đa lớp (Multilayer Perceptron - MLP)

Mạng nơ-ron nhân tạo (Artificial Neural Network - ANN) là một mô hình tính toán được lấy cảm hứng từ cấu trúc và chức năng của hệ thần kinh sinh học. Trong đó, mạng nơ-ron nhiều lớp (Multilayer Perceptron - MLP) là một dạng mạng nơ-ron phổ biến và quan trọng, có khả năng học các mối quan hệ phi tuyến giữa đầu vào và đầu ra thông qua nhiều tầng xử lý thông tin. Mạng nơ-ron nhiều lớp là một loại mạng nơ-ron truyền thẳng (feedforward neural network), trong đó các nơ-ron được tổ chức thành từng tầng, bao gồm: tầng đầu vào, một hoặc nhiều tầng ẩn, và tầng đầu ra. Mỗi tầng đều gồm nhiều nơ-ron, trong đó mỗi nơ-ron của một tầng bất kỳ đều được kết nối đầy đủ với tất cả các nơ-ron của tầng liền trước và tầng liền sau, tạo thành một mạng có kết nối dày đặc (fully connected).



Hình 10: Mạng nơ-ron nhân tạo nhiều lớp

Nguyên lý hoạt động:

Quá trình xử lý dữ liệu trong MLP được chia thành hai giai đoạn chính: lan truyền tiến (forward propagation) và lan truyền ngược (backpropagation). Trong giai đoạn lan truyền tiến, dữ liệu đầu vào được truyền từ tầng đầu vào qua các tầng ẩn tới tầng đầu ra. Mỗi nơ-ron thực hiện phép biến đổi tuyến tính với trọng số và hệ số dịch như sau:

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \quad a^{(l)} = \phi(z^{(l)})$$

Trong đó:

- $W^{(l)}$ là ma trận trọng số của tầng l
- $b^{(l)}$ là vector hệ số dịch (bias)
- $a^{(l-1)}$ là đầu ra của tầng trước
- ϕ là hàm kích hoạt phi tuyến

Sau khi đầu ra cuối cùng được tính toán, hàm mất mát (loss function) sẽ đo độ sai lệch giữa đầu ra dự đoán và nhãn thực tế. Hàm mất mát phổ biến trong bài toán phân loại là hàm cross-entropy, trong khi bài toán hồi quy thường dùng hàm MSE (Mean Squared Error).
Để cập nhật trọng số nhằm giảm sai số, thuật toán lan truyền ngược được sử dụng. Dựa trên đạo hàm của hàm mất mát đối với các trọng số, ta cập nhật từng trọng số theo hướng giảm gradient:

$$W^{(l)} \leftarrow W^{(l)} - \eta \frac{\partial L}{\partial W^{(l)}}$$

Trong đó η là tốc độ học (learning rate), quyết định mức độ điều chỉnh sau mỗi lần cập nhật. Việc cập nhật có thể được thực hiện bằng nhiều thuật toán tối ưu khác nhau như Gradient Descent, Stochastic Gradient Descent (SGD), hay Adam — một phương pháp hiện đại và phổ biến vì khả năng hội tụ nhanh và ổn định.

Vai trò của hàm kích hoạt và khả năng biểu diễn phi tuyến

Một điểm then chốt trong kiến trúc của MLP là việc sử dụng các hàm kích hoạt phi tuyến. Nếu toàn bộ mạng chỉ bao gồm các phép biến đổi tuyến tính, thì dù có nhiều tầng cũng không thể biểu diễn được hàm phi tuyến phức tạp. Hàm kích hoạt giúp mạng "bẻ cong"không gian đặc trưng, từ đó học được các mối quan hệ phức tạp trong dữ liệu.
Một số hàm kích hoạt phổ biến như:

- **Sigmoid:** thích hợp cho đầu ra nhị phân nhưng dễ bị "vanishing gradient".
- **Tanh:** cải tiến so với sigmoid do có đầu ra đối xứng quanh 0, nhưng vẫn bị mất gradient ở biên.
- **ReLU:** đơn giản và hiệu quả, giúp giải quyết vấn đề mất gradient, tuy nhiên có thể bị "chết nơ-ron"nếu giá trị đầu vào âm liên tục.
- **Leaky ReLU:** một biến thể của ReLU có thể giải quyết vấn đề chết nơ-ron bằng cách cho phép gradient nhỏ hơn 0 ở phía âm.

Nhờ vào các hàm kích hoạt này, MLP có thể xấp xỉ bất kỳ hàm liên tục nào, theo định lý xấp xỉ phổ quát (Universal Approximation Theorem), nếu được cung cấp đủ nơ-ron và dữ liệu huấn luyện phù hợp.
Trong khi các phương pháp như hồi quy tuyến tính hoặc cây quyết định có thể hoạt động tốt với các mối quan hệ đơn giản hoặc có cấu trúc rõ ràng, thì MLP (Multi-Layer Perceptron) lại có khả năng học được các mối quan hệ phi tuyến mà các phương pháp truyền thống không thể mô phỏng. Điều này đặc biệt quan trọng trong bài toán phân loại tắc nghẽn giao thông, vì tình trạng giao thông thường bị ảnh hưởng bởi nhiều yếu tố phức tạp như:

- Thời gian trong ngày
- Vị trí địa lý
- Loại đường và mật độ phương tiện
- Thời tiết
- Sự kiện đặc biệt hoặc tai nạn

Những yếu tố này thường không có mối quan hệ tuyến tính rõ ràng với mức độ tắc nghẽn, khiến các mô hình đơn giản khó nắm bắt được.
Với khả năng xấp xỉ các hàm phi tuyến, MLP có thể:

- Học được các mẫu dữ liệu phức tạp và linh hoạt hơn
- Tự động phát hiện các mối quan hệ tiềm ẩn giữa các đặc trưng mà không cần giả định trước
- Tăng độ chính xác trong phân loại các mức độ tắc nghẽn (ví dụ: không tắc, tắc nhẹ, tắc nặng)

Khác với các mô hình tuyến tính, MLP không yêu cầu giả định về hình thức mối quan hệ giữa đầu vào và đầu ra. Điều này giúp mô hình thích ứng tốt hơn với sự biến động và tính không ổn định của dữ liệu giao thông thực tế, đặc biệt trong các tình huống bất thường như lễ hội, tai nạn, hoặc thay đổi thời tiết đột ngột.

4.2.1.3 Đánh giá mô hình bằng F1-score

F1-score là thước đo hiệu suất phù hợp trong các bài toán phân loại với dữ liệu không cân bằng, vì nó cân bằng giữa Precision (độ chính xác) và Recall (độ bao phủ). Việc sử dụng F1-score giúp phản ánh đúng khả năng phân biệt giữa các lớp, đặc biệt khi lớp thiểu số có vai trò quan trọng.

Các bước đánh giá:

- Huấn luyện mô hình
  - XGBoost:
    - Cấu hình scale_pos_weight để ưu tiên lớp thiểu số.
    - Tối ưu hóa theo hàm mất mát phân loại (log loss).
  - MLP (Multi-Layer Perceptron):
    - Cần chuẩn hóa dữ liệu đầu vào.
    - Sử dụng class_weight để cân bằng ảnh hưởng giữa các lớp.
    - Điều chỉnh số tầng ẩn và số lượng neuron để phù hợp với độ phức tạp dữ liệu.

- Dự đoán và đánh giá
  - Sử dụng tập kiểm tra để đánh giá mô hình.
  - Tính F1-score theo từng lớp hoặc trung bình có trọng số (weighted average) để phản ánh đúng hiệu suất tổng thể.

```
1    from sklearn.metrics import f1_score
2
3    f1 = f1_score(y_true, y_pred, average='weighted')
4    print("F1-score:", f1)
5
```

5 Kết quả

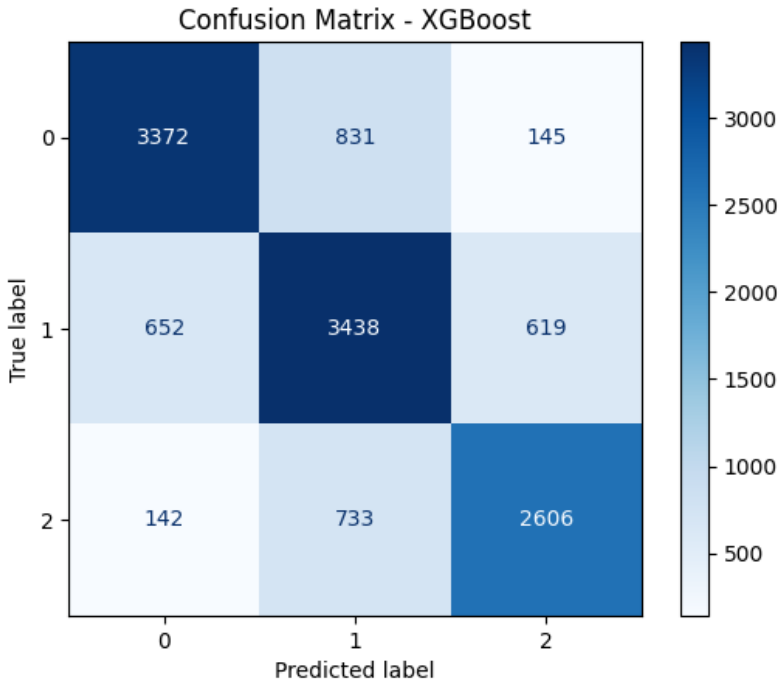
5.1 Đánh giá kết quả mô hình

5.1.1 XGBoost

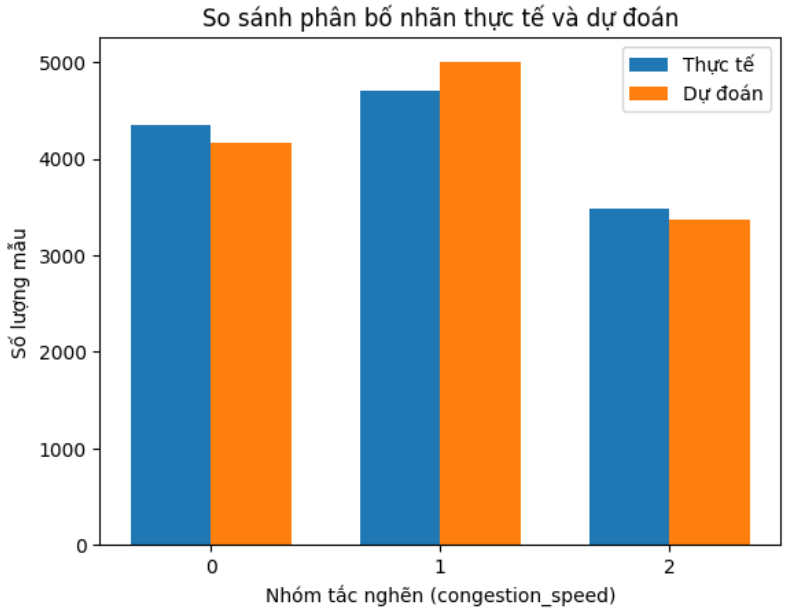
```
Accuracy: 0.7665827498360869
[[116000  24102   3506]
 [ 21217 120823  16938]
 [  4362  23149  69505]]
              precision    recall  f1-score   support

           0       0.82      0.81      0.81    143608
           1       0.72      0.76      0.74    158978
           2       0.77      0.72      0.74     97016

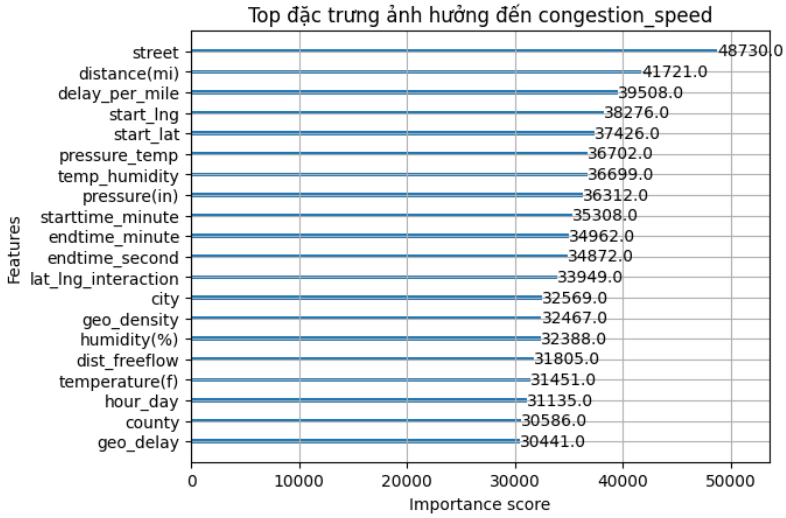
    accuracy                           0.77    399602
   macro avg       0.77      0.76      0.77    399602
weighted avg       0.77      0.77      0.77    399602
```



Hình 11: Confusion Matrix XGBoost



Hình 12: Biểu đồ so sánh giữa Dự đoán và thực tế XGBoost



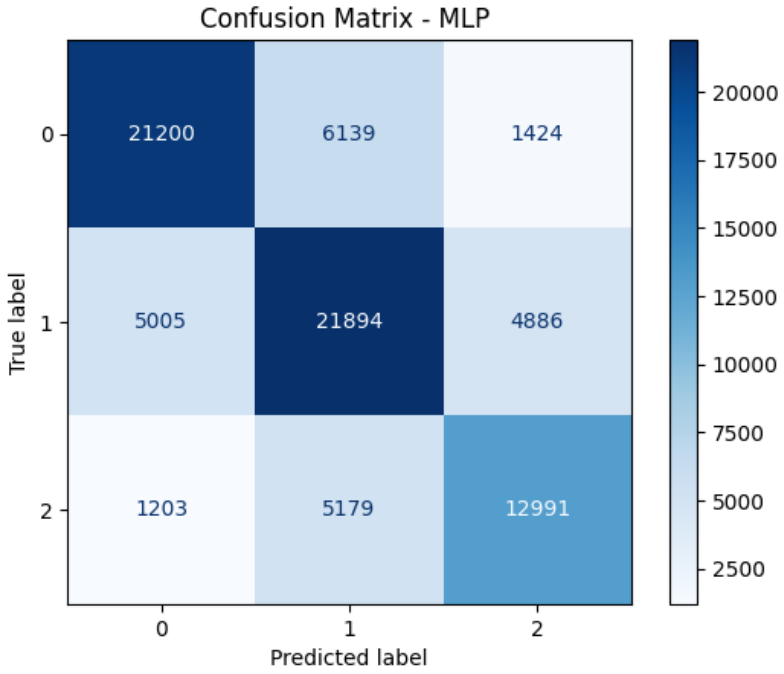
Hình 13: Các đặc trưng ảnh hưởng congestion speed

5.1.2 MLP

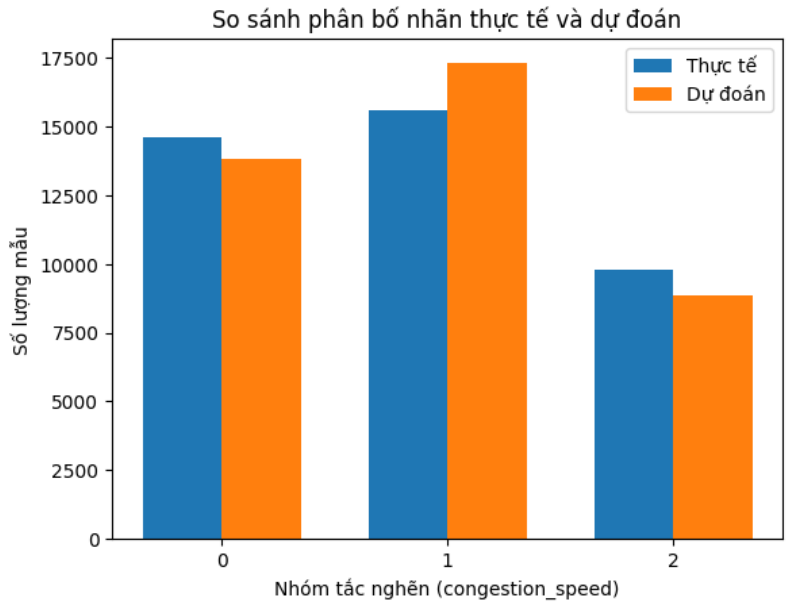
```
precision    recall  f1-score   support

           0       0.78      0.80      0.79    143608
           1       0.77      0.56      0.65    158978
           2       0.61      0.86      0.71     97016
```

```
6
7     accuracy                           0.72    399602
8    macro avg       0.72      0.74      0.72    399602
9 weighted avg       0.73      0.72      0.72    399602
10
11 [[115523  16956  11129]
12  [ 28560  88922  41496]
13  [  3917   9952  83147]]
```



Hình 14: Confusion Matrix MLP



Hình 15: Biểu đồ so sánh giữa Dự đoán và thực tế MLP

Đánh giá tổng quan và so sánh hiệu suất mô hình:

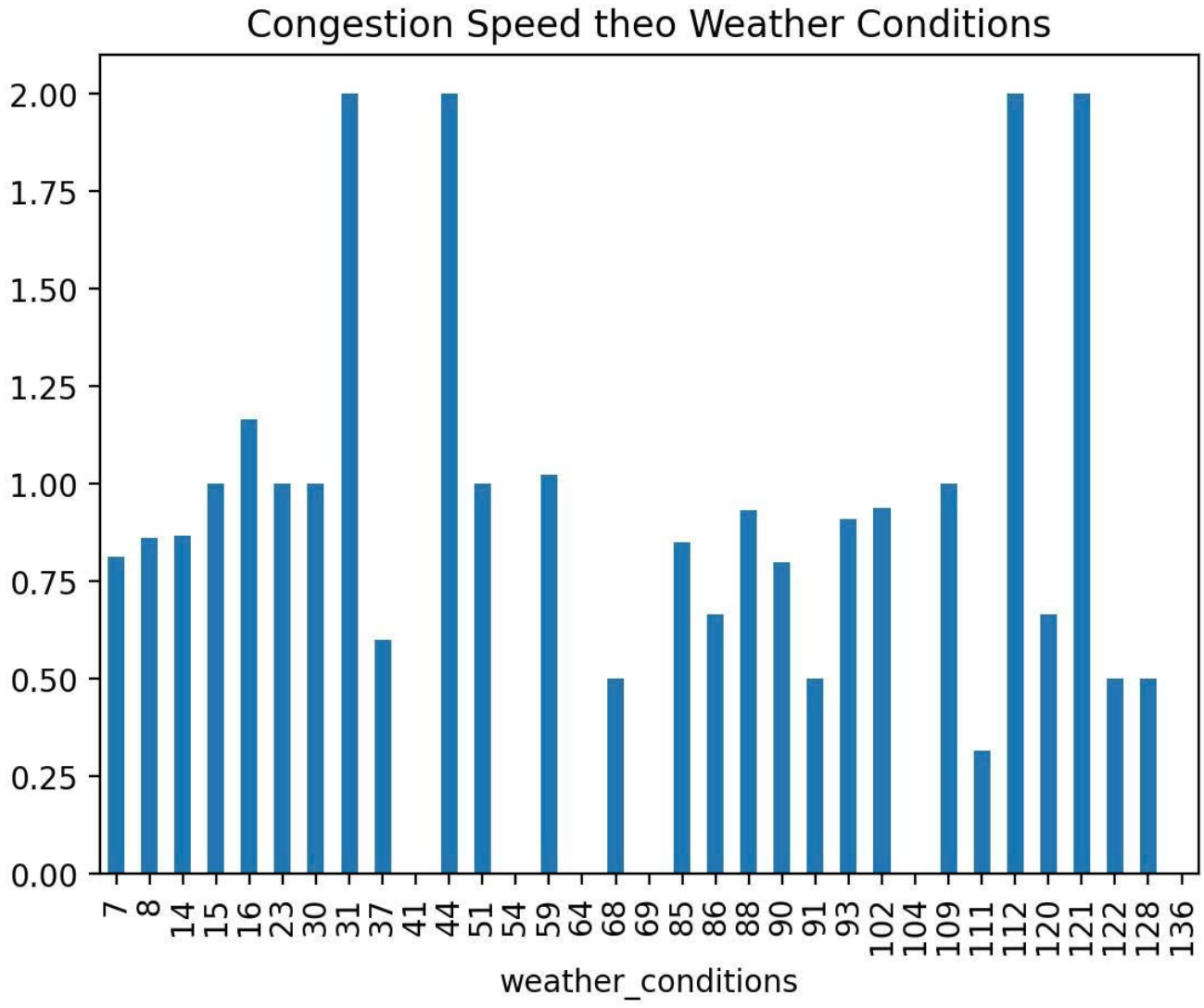
Dựa trên kết quả thực nghiệm, mô hình XGBoost thể hiện sự vượt trội toàn diện so với mô hình MLP trên tập dữ liệu kiểm thử. Về mặt hiệu suất tổng thể, XGBoost đạt độ chính xác (Accuracy) là 76.66%, cao hơn đáng kể so với mức 72% của MLP. Điều này cho thấy kiến trúc Tree-based của XGBoost đang khớp tốt hơn với đặc trưng của dữ liệu này so với mạng nơ-ron MLP hiện tại.

Sự khác biệt rõ rệt nhất giữa hai mô hình nằm ở tính ổn định giữa các lớp dữ liệu. XGBoost duy trì được sự cân bằng tốt giữa độ chính xác (Precision) và độ nhạy (Recall) cho cả ba lớp (0, 1, và 2), với chỉ số F1-score đều dao động ở mức cao (từ 0.74 đến 0.81). Trong khi đó, MLP thể hiện sự mất cân đối nghiêm trọng. Cụ thể, đối với Lớp 1 (lớp chiếm đa số), MLP hoạt động rất kém với chỉ số Recall chỉ đạt 0.56, đồng nghĩa với việc mô hình này bỏ sót tới 44% số lượng mẫu thực tế của Lớp 1.

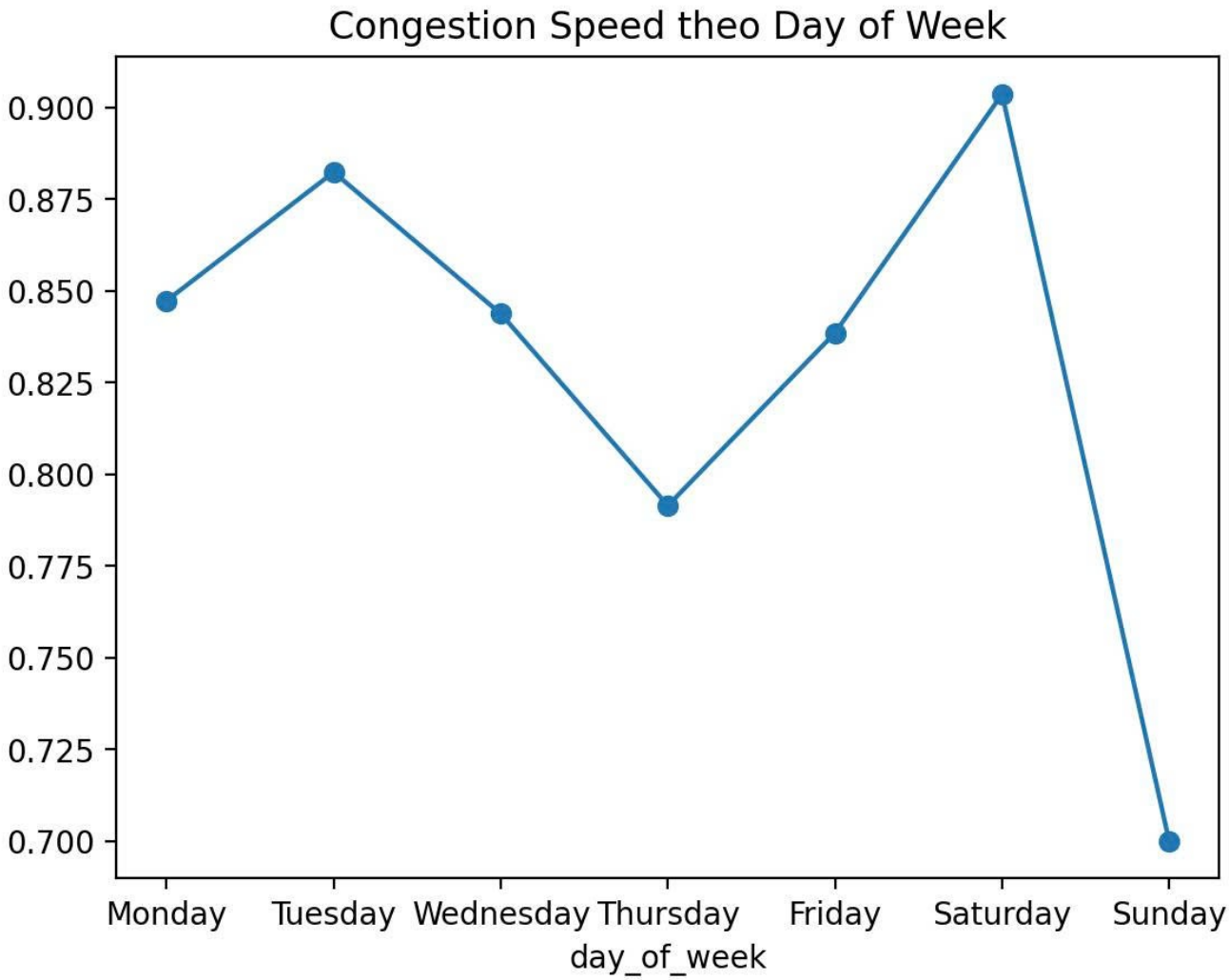
Nguyên nhân chính dẫn đến sự sụt giảm hiệu suất của MLP được thể hiện rõ qua Ma trận nhầm lẫn (Confusion Matrix). Mô hình MLP có xu hướng dự đoán nhầm một lượng lớn dữ liệu từ Lớp 1 sang Lớp 2 (cụ thể là 41,496 mẫu bị nhầm, so với chỉ 16,938 mẫu ở XGBoost). Hệ quả là tại Lớp 2, mặc dù MLP có độ nhạy (Recall) rất cao (0.86) – tức là bắt được hầu hết các trường hợp của Lớp 2 – nhưng độ chính xác (Precision) lại sụt giảm xuống còn 0.61 do chứa quá nhiều mẫu dương tính giả (False Positives) từ

Lớp 1 chuyển sang.

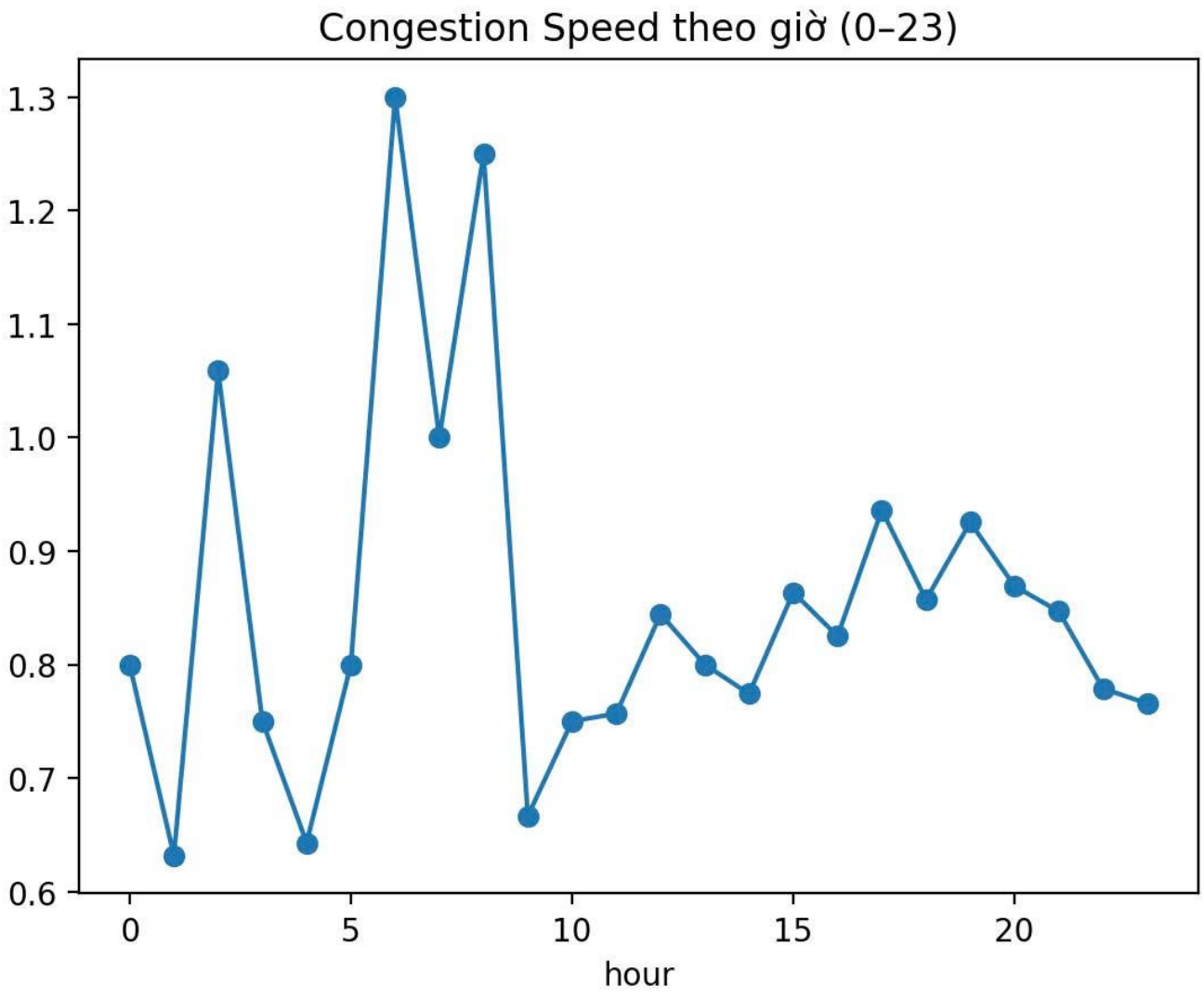
Kết luận: Mô hình XGBoost là lựa chọn tối ưu hơn trong trường hợp này nhờ khả năng phân lớp chính xác và đồng đều, không bị thiên kiến (bias) quá mức vào một lớp cụ thể nào. Mô hình MLP hiện tại đang gặp vấn đề trong việc phân tách ranh giới giữa Lớp 1 và Lớp 2, dẫn đến việc hy sinh độ chính xác của lớp đông nhất (Lớp 1) để tối đa hóa khả năng nhận diện Lớp 2.



Hình 16: Tốc độ tắc nghẽn theo điều kiện thời tiết



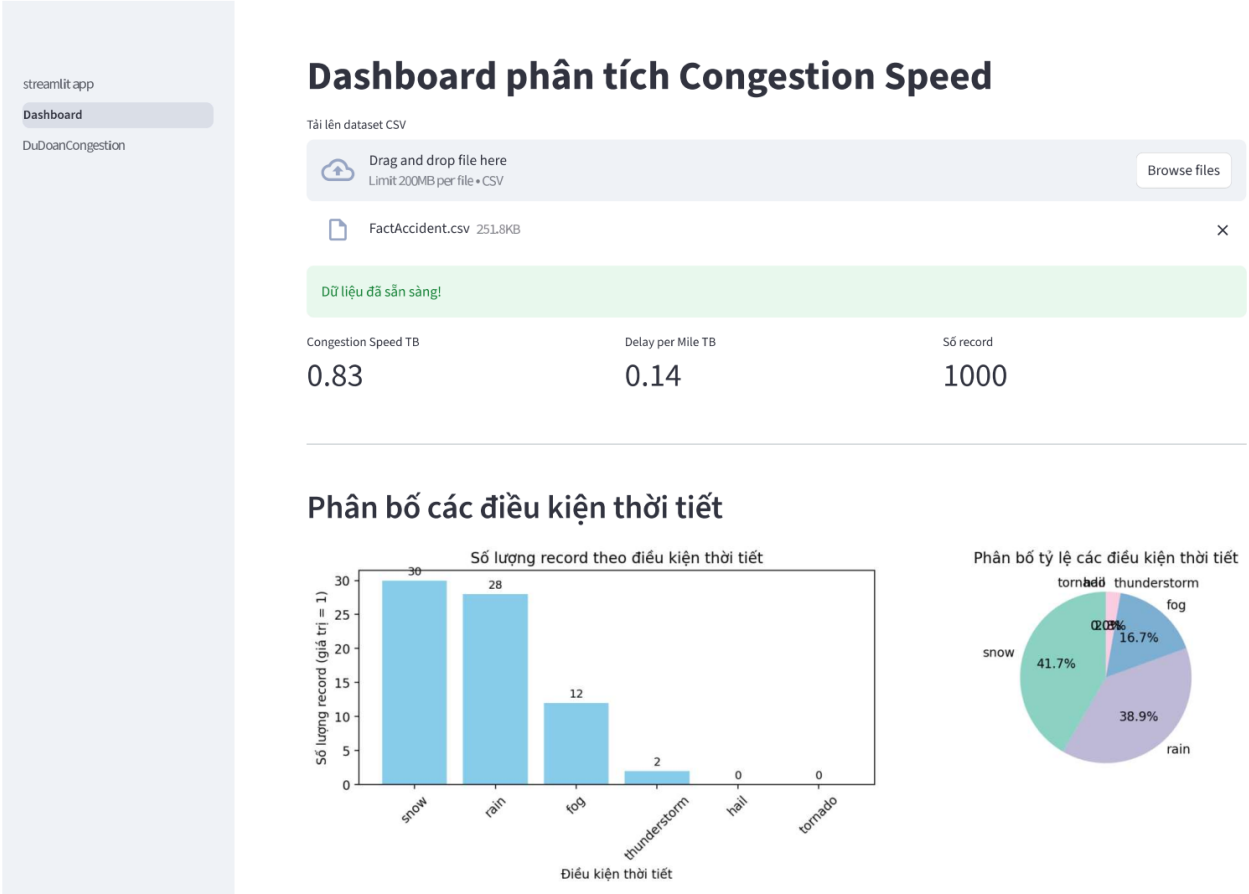
Hình 17: Tốc độ tắc nghẽn theo các ngày trong tuần



Hình 18: Tốc độ tắc nghẽn theo giờ

5.1.3 Giao diện Streamlit

Hệ thống được triển khai thành ứng dụng web thông qua framework Streamlit, cung cấp giao diện trực quan và thân thiện với người dùng.



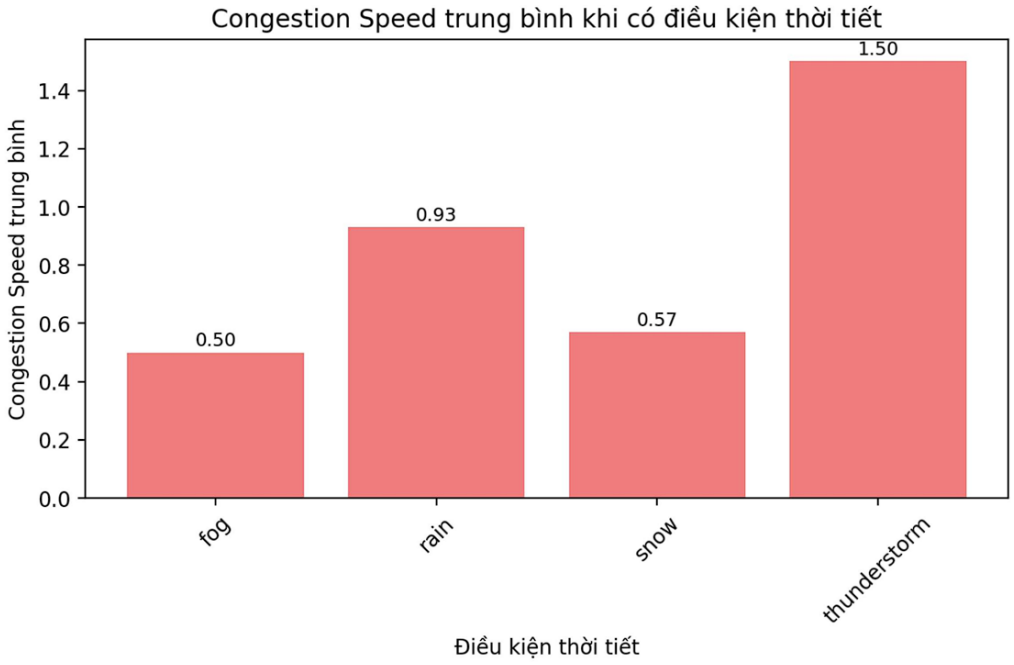
Hình 19: Màn hình tổng quan dữ liệu

Thống kê chi tiết

	Weather Condition	Count	Percentage
0	clear	937	93.7
1	snow	30	3
2	rain	28	2.8
3	fog	12	1.2
4	thunderstorm	2	0.2
5	hail	0	0
6	tornado	0	0

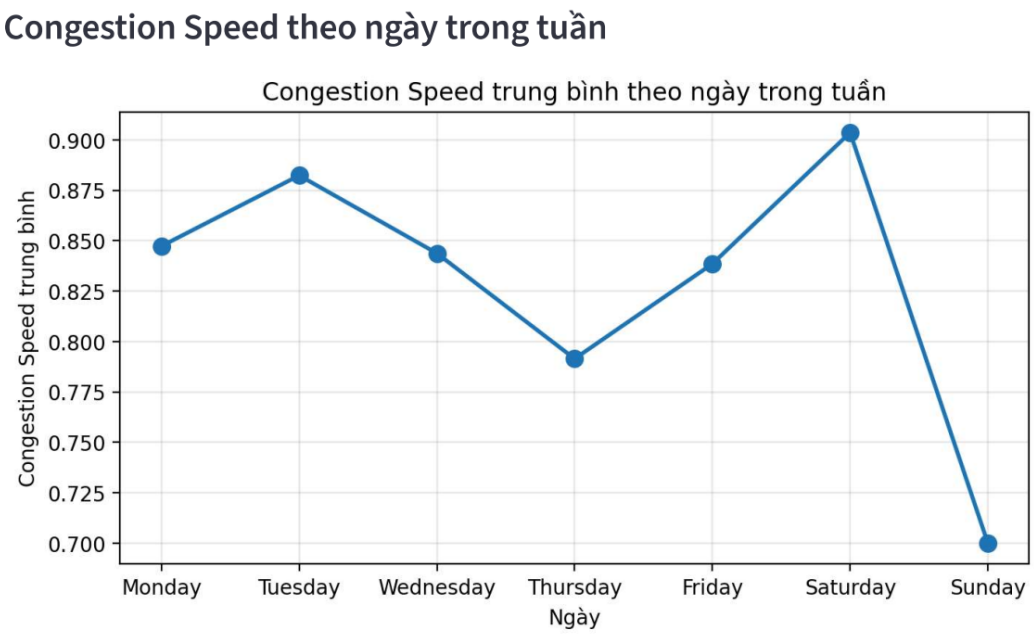
Hình 20: Thống kê chi tiết

Congestion trung bình theo điều kiện thời tiết

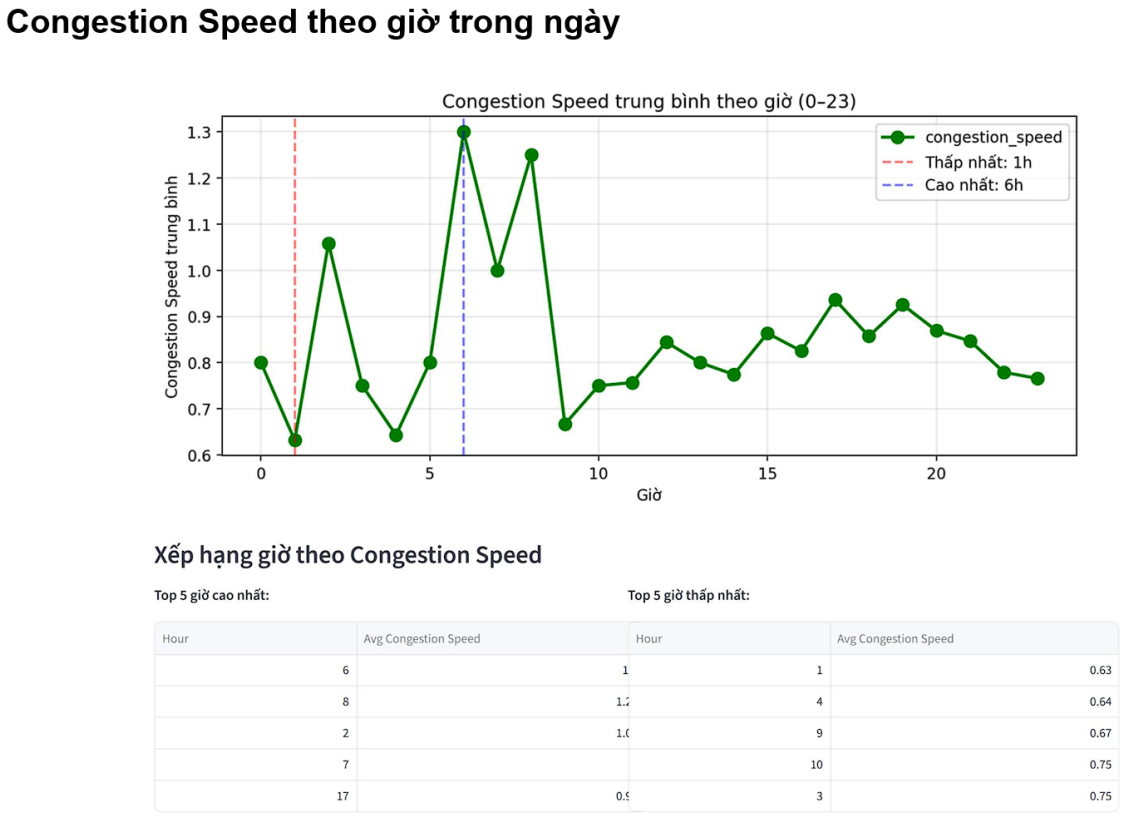


	Weather Condition	Avg Congestion Speed	Count
0	fog	0.5	12
1	rain	0.93	28
2	snow	0.57	30
3	thunderstorm	1.5	2

Hình 21: Tốc độ tắc nghẽn trung bình theo điều kiện thời tiết



Hình 22: Tốc độ tắc nghẽn theo các ngày trong tuần



Hình 23: Tốc độ tắc nghẽn theo giờ trong ngày

6 Kết luận và hướng phát triển

6.1 Kết luận

Dự án đã hoàn thành mục tiêu xây dựng một quy trình Kho dữ liệu và Khai phá dữ liệu toàn diện cho bài toán giao thông đô thị.

1. **Về mặt công nghệ:** Nhóm đã tích hợp thành công dữ liệu đa nguồn, xử lý các vấn đề về nhiễu và mất cân bằng dữ liệu, đồng thời so sánh hiệu quả của hai phương pháp học máy tiên tiến là XGBoost và MLP.

2. **Về mặt kết quả:** Mô hình XGBoost cho thấy sự vượt trội với độ chính xác 77% và F1-Score 0.77, chứng minh tính khả thi trong việc hỗ trợ ra quyết định.

3. **Về mặt tri thức:** Kết quả phân tích đặc trưng (Feature Importance) đã chỉ ra rằng hạ tầng giao thông (đặc điểm tuyến đường, khoảng cách di chuyển) có tác động lớn hơn yếu tố thời tiết đối với tình trạng tắc nghẽn, cung cấp góc nhìn quan trọng cho các nhà quy hoạch đô thị.

6.2 Hạn chế

Hạn chế nội tại của đề tài: Mặc dù đạt được kết quả khả quan, mô hình hiện tại vẫn tồn tại một số hạn chế về mặt kỹ thuật:

1. **Thiếu tính ngữ cảnh chuỗi thời gian:** Mô hình hiện tại coi các sự kiện là độc lập (independent instances). Tuy nhiên, tắc nghẽn giao thông có tính kế thừa cao (ùn tắc ở phút thứ t sẽ ảnh hưởng mạnh đến phút thứ $t+1$). Việc sử dụng mô hình dạng bảng (Tabular) chưa khai thác hết tính phụ thuộc thời gian này.

2. **Sai số ở các lớp biên:** Phân tích Confusion Matrix cho thấy mô hình vẫn gặp khó khăn trong việc phân biệt rạch ròi giữa mức độ "Tắc nhẹ"và "Tắc nghiêm trọng", có thể dẫn đến các cảnh báo sai (False Positives) hoặc bỏ sót (False Negatives) trong các tình huống nhạy cảm.

6.3 Hướng phát triển

Để khắc phục các hạn chế trên và nâng cao tính thực tiễn của hệ thống DSS, nhóm đề xuất các hướng phát triển trong tương lai:

1. **Chuyển đổi sang mô hình Chuỗi thời gian (Time-series Modeling):** Thay vì sử dụng các mô hình học giám sát truyền thống, cần chuyển sang các kiến trúc Deep Learning chuyên biệt cho dữ liệu chuỗi như LSTM (Long Short-Term Memory), GRU hoặc Transformer. Các mô hình này có khả năng "nhớ"trạng thái giao thông của các khung giờ trước đó để dự báo tương lai chính xác hơn

2. **Làm giàu dữ liệu (Data Enrichment):** Tắc nghẽn không chỉ do hạ tầng hay thời tiết. Cần tích hợp thêm các nguồn dữ liệu sự kiện xã hội (Lịch thi đấu bóng đá, Liveshow ca nhạc, Lễ hội...) và dữ liệu cảm xúc từ mạng xã hội (Social Listening) để dự báo các điểm nóng tắc nghẽn bất thường (Anomalies) mà dữ liệu lịch sử không ghi nhận được.

3. **Tối ưu hóa cho triển khai biên (Edge Deployment):** Áp dụng các kỹ thuật Nén mô hình (Model Quantization) hoặc Cắt tỉa (Pruning) để giảm kích thước và độ phức tạp tính toán của mô hình mà không làm giảm đáng kể độ chính xác. Điều này cho phép mô hình chạy mượt mà trên các thiết bị IoT tại nút giao thông, giải quyết bài toán về độ trễ thời gian thực đã nêu.

7 Ràng buộc bổ sung và hạn chế thực tế

Mặc dù mô hình XGBoost của nhóm đã đạt độ chính xác khả quan (77%) trên tập dữ liệu thử nghiệm, việc chuyển đổi từ một mô hình nghiên cứu (Research) sang một hệ thống điều hành giao thông thực tế (Production) sẽ đối mặt với những thách thức phi kỹ thuật và hạ tầng mà bộ dữ liệu tĩnh hiện tại chưa phản ánh được. Dưới đây là 3 ràng buộc thực tế quan trọng nhất:

7.1 Ràng buộc về độ trễ thời gian thực (Real-time Latency Constraint)

Trong môi trường thí nghiệm, dữ liệu được xử lý theo lô (Batch processing), thời gian dự báo cho hàng triệu dòng dữ liệu có thể mất vài phút mà không ảnh hưởng đến kết quả. Tuy nhiên, trong hệ thống Giao thông thông minh (ITS):

- **Yêu cầu:** Hệ thống cần đưa ra quyết định điều khiển đèn tín hiệu hoặc cảnh báo tài xế với độ trễ cực thấp (thường dưới 200ms đến 1 giây).

- **Thách thức:** Các mô hình phức tạp như MLP (Deep Learning) hoặc Ensemble lớn (XGBoost với 800 cây quyết định) đòi hỏi tài nguyên tính toán lớn. Khi triển khai trên các thiết bị biên (Edge Devices) như camera giao thông hay hộp điều khiển tín hiệu (có CPU/RAM hạn chế), thời gian suy diễn (Inference time) có thể bị kéo dài, dẫn đến "độ trễ quyết định", khiến giải pháp không còn tác dụng tức thời đối với dòng xe đang lưu thông tốc độ cao.

7.2 Tính đầy đủ và Tin cậy của hạ tầng cảm biến (Sensor Reliability & Availability)

Mô hình hiện tại được huấn luyện trên một bộ dữ liệu đã được làm sạch và nội suy kỹ lưỡng (Cleaned Dataset).

- **Thực tế:** Các cảm biến IOT ngoài trời chịu tác động khắc nghiệt của môi trường (nhiệt độ cao, mưa bão, bụi bẩn) dẫn đến việc thường xuyên bị hỏng hóc, mất kết

nối đường truyền hoặc cảm biến bị trôi (Sensor drift - sai số tăng dần theo thời gian).

- **Rủi ro:** Khi một hoặc nhiều cảm biến đầu vào bị mất tín hiệu đột ngột (ví dụ: mất dữ liệu Precipitation do hỏng cảm biến mưa), mô hình có thể bị lỗi (crash) hoặc đưa ra dự báo sai lệch hoàn toàn nếu hệ thống thực tế không có cơ chế chịu lỗi (Fault-tolerance) và cơ chế điền khuyết dữ liệu thời gian thực (Real-time Imputation) mạnh mẽ như trong môi trường Offline.

7.3 Rào cản về Bảo mật và Quyền riêng tư (Privacy & Regulatory Compliance)

Dữ liệu hiện tại bao gồm tọa độ GPS chính xác (Start_Lat, Start_Lng) của các sự kiện, đây là yếu tố quan trọng giúp mô hình học được đặc trưng địa lý (geo_density).

- **Thực tế:** Việc thu thập và lưu trữ dữ liệu vị trí chi tiết của người dân chịu sự kiểm soát nghiêm ngặt của các luật định (như GDPR tại Châu Âu hay Nghị định 13/2023/NĐ-CP tại Việt Nam).

- **Thách thức:** Để tuân thủ pháp luật, dữ liệu thực tế thường phải trải qua quá trình ẩn danh hóa (Anonymization), ví dụ như làm mờ vị trí (giảm độ chính xác GPS) hoặc chỉ thu thập dữ liệu tổng hợp. Việc này sẽ làm giảm độ chính xác của các đặc trưng địa lý, từ đó làm suy giảm hiệu suất dự báo của mô hình so với kết quả trong phòng thí nghiệm. Bài toán đặt ra là phải cân bằng (Trade-off) giữa độ chính xác của thuật toán và quyền riêng tư của người dùng.

Tài liệu

[1] S. Moosavi, M. H. Samavatian, S. Parthasarathy, and R. Ramnath, "A Countrywide Traffic Accident Dataset", 2019. [Online]. Available: https://arxiv.org/abs/1906.05409.

[2] S. Moosavi, "US Traffic Congestions (2016-2022)", Kaggle, 2022. [Online]. Available: https://www.kaggle.com/datasets/sobhanmoosavi/us-traffic-congestion-dataset.

[3] INRIX Research, "INRIX Global Traffic Scorecard 2024-2025", 2025. [Online]. Available: https://inrix.com/scorecard/.

[4] W. H. Inmon, *Building the Data Warehouse*, 4th ed. Indianapolis, IN: Wiley, 2005.

[5] R. Kimball and M. Ross, *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*, 3rd ed. Indianapolis, IN: Wiley, 2013.

[6] E. Turban, R. Sharda, and D. Delen, *Decision Support and Business Intelligence Systems*, 9th ed. Upper Saddle River, NJ: Pearson Education, 2010.

[7] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System", in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, San Francisco, CA, USA, 2016, pp. 785–794.

[8] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA: MIT Press, 2016.

[9] Hitachi Vantara, "Pentaho Data Integration Documentation", 2025. [Online]. Available: https://help.hitachivantara.com/.

[10] Streamlit Inc., "Streamlit Documentation", 2025. [Online]. Available: https://docs.streamlit.io/.